Chip Hippo

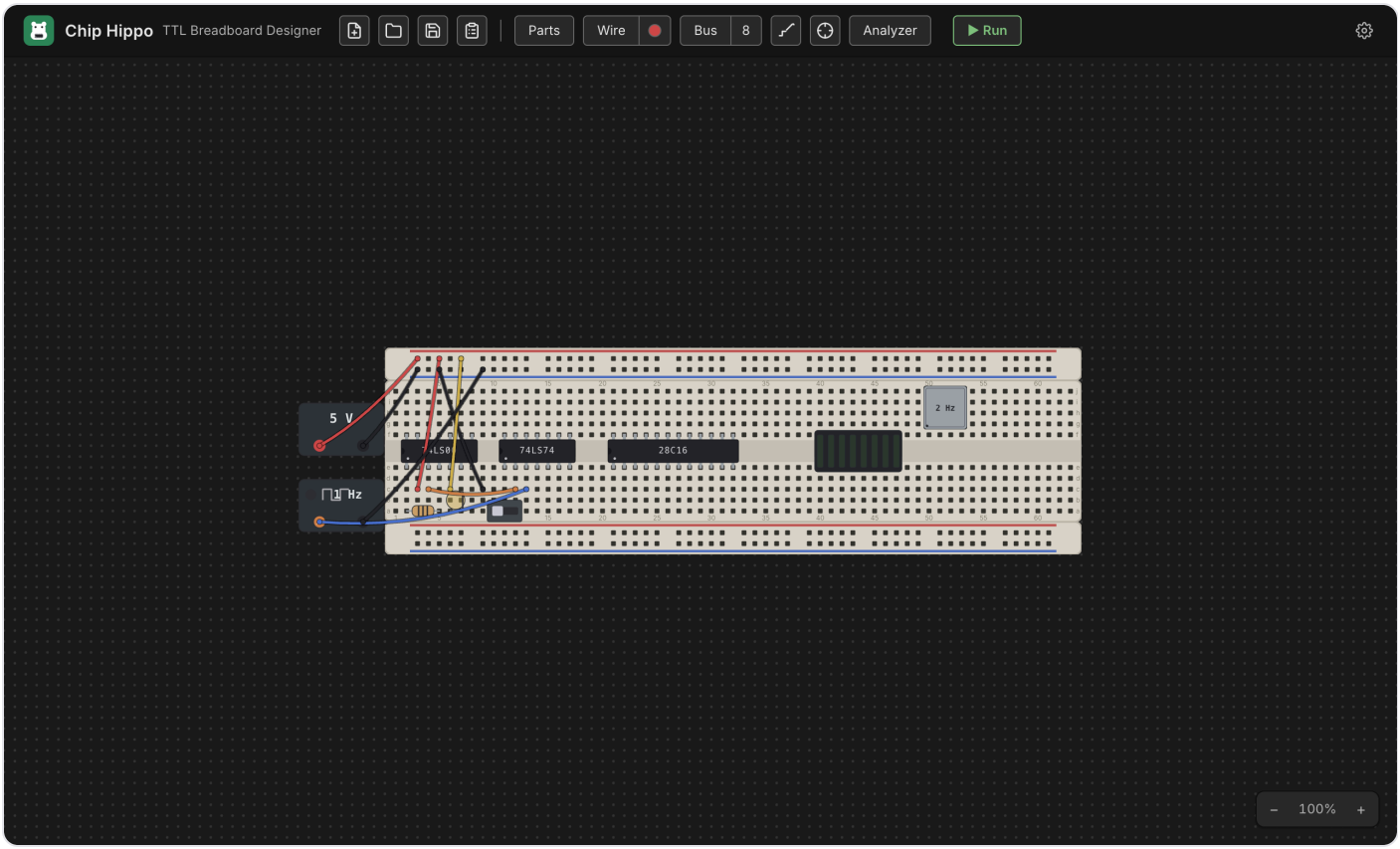
User Guide

Version 0.8.0 · July 27, 2026

Contents

Chip Hippo User Guide

Chip Hippo is a desktop app for designing and simulating **74xx TTL logic circuits** on virtual solderless breadboards. Place a breadboard on the infinite desk, populate it with 74xx DIP chips, wires, switches, LEDs, and power sources, then press **Run** — a simulation engine traces electricity from every power source, resolves each electrical net, and ripples changes through the circuit exactly like a real breadboard would.



What you can build

Anything a 74xx TTL breadboard bench can hold: combinational logic (gates, decoders, multiplexers), sequential logic (flip-flops, counters, shift registers), a free-running or manually stepped clock, and small memory circuits backed by ROM/RAM chips — all wired across Full/Half/Tiny breadboards, powered at 3 V / 5 V / 12 V, and watched settle live.

Highlights

- **Breadboard-accurate placement** — chips, wires, and discretes snap to the real 0.1 in hole grid, with the same row-half/trench/rail tie-point rules as a physical board.

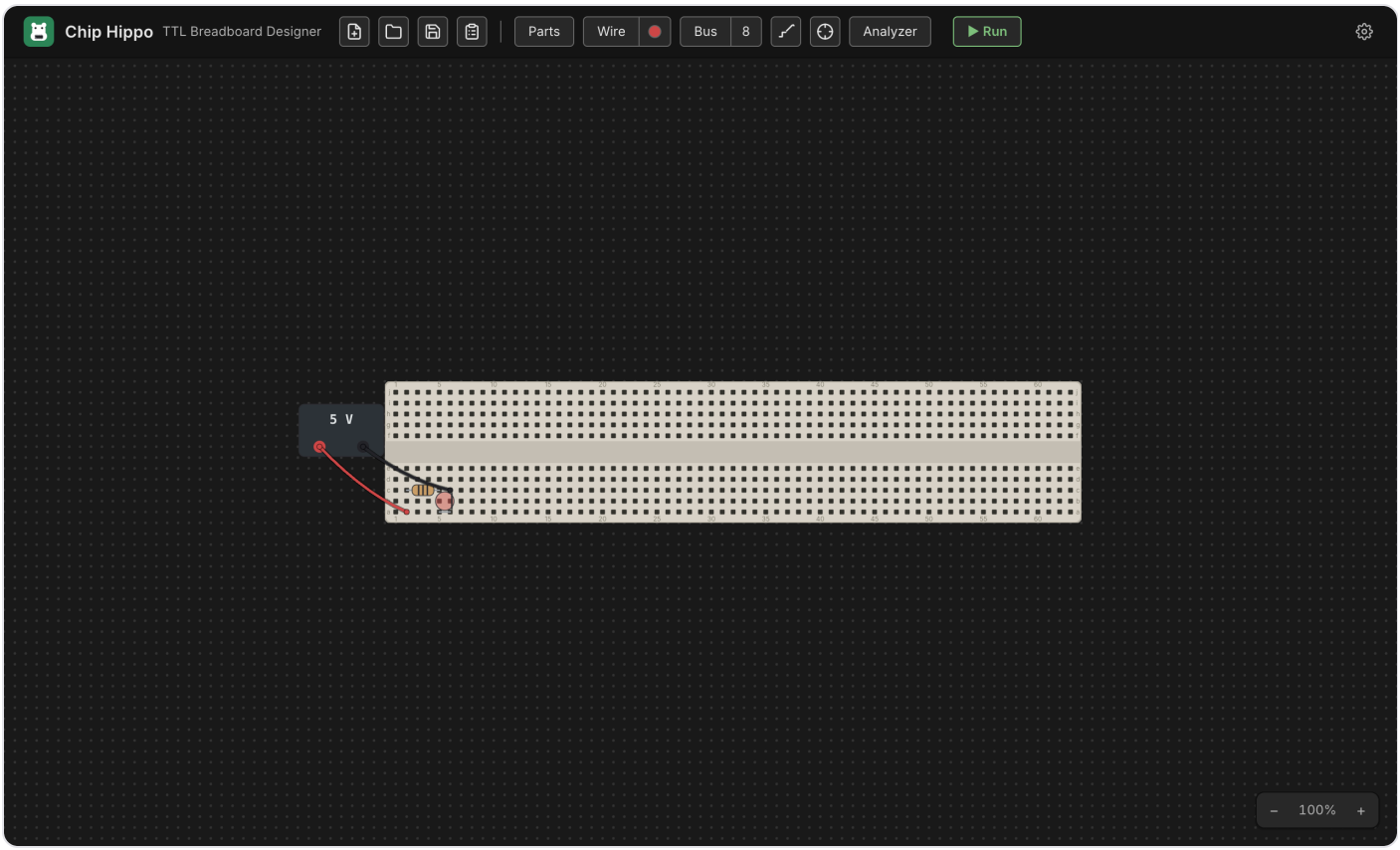
- **Live simulation** — Run the circuit and watch LEDs light, chip health badges report power/damage state, and switches drive the circuit live.
- **Deep inspection tools** — a connectivity **probe**, **net names/labels**, and a **logic analyzer** for capturing waveforms.
- **Memory chips with a hex inspector** — file-backed ROM/EEPROM images you program with an in-app programmer, plus a live hex/ASCII viewer.
- **A derived schematic view** — press `Tab` to flip between the physical breadboard and a logical diagram of chip symbols, routed named nets, and bus lines, kept in sync with the desk.
- **Build guide & BOM export**, and **undo/redo** across every edit, with an always-autosaved working document.

Table of contents

- Keyboard Shortcuts — every shortcut in one place.

Getting Started

This page walks through building the smallest possible circuit — a breadboard, a power supply, and an LED — from a completely empty desk to a lit LED. It takes about five minutes and introduces the tools you'll use for every larger circuit afterward.



Launch Chip Hippo

Open the installed app like any other desktop program (or, if you're running from source, `make debug`). Chip Hippo opens on an **infinitely pannable, zoomable desk** — drag to pan, scroll or pinch to zoom, and the header's zoom controls (bottom-right of the desk) reset or step the view. The first time you launch, the desk is empty and shows a hint: *"Open the parts tray and add a breadboard to get started."*

Open the parts palette

The palette is a tray docked to the left of the desk, and it carries its own open/close control rather than a toolbar button. When it is shut, a small chevron flap sits against the desk's left edge — click it (or press `Ctrl+P` / `⌘P`) to slide the tray out. The matching chevron in the tray's top-right corner shuts it again, back to nothing.

At the top of the tray is the **board selector** (Full / Half / Tiny breadboards, plus loose strips); below that, chips and components grouped by function, with a filter box to search by name.

Add a breadboard

In the palette's **BOARDS** section, click **Full-size** (830 tie points) — the default recommendation for a first circuit; it has the most room and both power rails already attached. A translucent ghost of the whole kit follows your cursor. Move it over open desk space and click to drop it.

The Full kit places three strips as one rigid group: a power rail, the pin-board, and a second power rail — exactly like a real solderless breadboard. You can drag the whole board later by its body, or reopen the palette any time to add more.

Add power

Still in the palette, open the **COMPONENTS ▸ Power** group and click **Power supply**. A small PSU brick ghost follows your cursor — click open desk space near the breadboard to place it. A PSU brick isn't seated on the board; it's a free-standing part with two wireable terminals, **+** and **−**.

Right-click the placed PSU and choose **Properties...** to confirm it's set to **5 V** (new PSUs default to 5 V, but 3 V and 12 V are also there — 12 V is enough to damage a chip, so save it for parts rated to take it). **Delete Component**, further down the same context menu, removes the PSU if you want to start over.

Add an LED

Back in the palette, open **COMPONENTS ▸ LEDs** and click **LED**. A small LED ghost follows your cursor, already colored with your "Default LED color" setting (Settings ▸ Appearance — red out of the box). Move it over a free row of breadboard holes and click to seat it — an LED's two legs land in adjacent holes on the same row, anode first. (Press `F` while the ghost is in hand to flip which leg is the anode, or `R` to stand it up on two free ends instead of a footprint row.) To change an LED's color afterward, right-click it and choose **Properties...**.

Chip Hippo's LED is idealized — no series resistor is required to protect it, so you can wire it straight to a supply.

Wire it up

Press `W` (or click **Wire** in the toolbar) to arm the wire tool. Click a free hole or terminal to anchor the first end, then click a second free point to lay the wire — a rubber-band preview tracks your cursor between

clicks, and it turns red over an illegal target. The tool stays armed, so you can lay several wires in a row without re-arming it; press `Esc` to cancel a pending wire, or `Esc` /click **Wire** again to disarm the tool.

Lay two wires:

1. **PSU `+` → the LED's anode hole** (or a rail hole in the same row/column as the anode, then the anode's row completes the path through the board).
2. **PSU `–` → the LED's cathode hole**, the same way.

The active wire color is shown as a dot on the **Wire** button; click the dot to open the color palette and pick a different one — the color you pick stays active for every wire you lay afterward, so use different colors for positive and return paths if you want them easy to tell apart at a glance.

Run it

Press **Run** in the header toolbar (or `Space`) to start the simulation. The engine traces power from the PSU, resolves the electrical net at every hole, and settles the circuit — the LED should light immediately, since anode-high through cathode-low is exactly what a driven LED needs.

While running, editing is locked (you can't move parts or lay new wires), but the connectivity **probe**, the **logic analyzer**, and the **build guide** all stay live. Press **Stop** (or `Space` again) to freeze the simulation and return to editing.

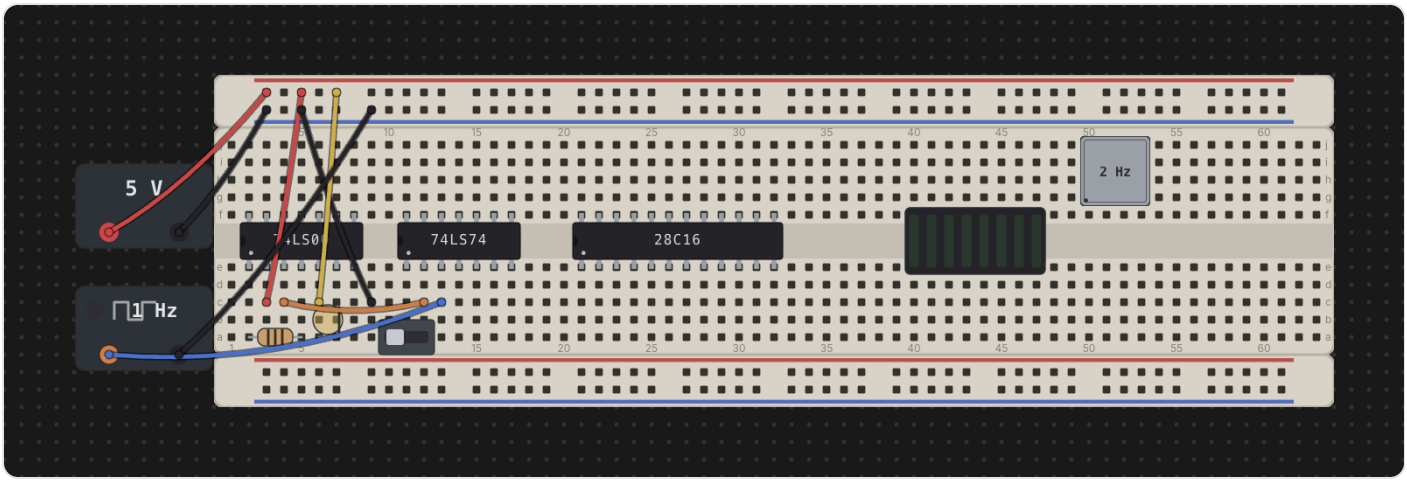
If the LED doesn't light, double-check both wires land on the correct terminal (`+` / `–`) and hole, and that the PSU is on and not showing a damage badge.

What next

- **The Desk & Breadboards** — pan/zoom, breadboard kits, strips and rails, snapping and grouping multiple boards together.
- **Chips & Components** — the parts palette in depth: DIP chips, discretes, placement, and rotation.
- **Wiring, Nets & Buses** — the wire tool in depth, cross-board wires, colors, and multi-bit buses.
- **Power & Clock Sources** — PSU voltage and the 12 V damage rule, plus clock sources for sequential circuits.
- **Running a Simulation** — Run/Pause/Step, the settle model, and how live views (LEDs, chip badges) work.

The Desk & Breadboards

The desk is Chip Hippo's infinite, pannable, zoomable workspace — the place every board, chip, wire, and power source lives. This page covers how to move around the desk and how breadboards actually work under the hood: they are not single parts but individual **strips** — a pin-board plus power rails — that snap and mate together the way real boards do on a bench.



Pan, zoom & fit to screen

The desk has no edges — pan and zoom to work at whatever scale suits the circuit in front of you.

- **Pan**: click-drag an empty patch of desk, or two-finger scroll on a trackpad.
- **Zoom**: scroll/pinch to zoom, centered on the pointer, or use the **+** / **−** buttons in the zoom control. Keyboard: `Cmd/Ctrl+=` (zoom in), `Cmd/Ctrl+−` (zoom out), `Cmd/Ctrl+0` (reset to 100%).
- **Fit to screen**: `Cmd/Ctrl+F` frames everything currently on the desk in the viewport — the fastest way to find your circuit after panning away.
- **Zoom out full**: `Cmd/Ctrl+Shift+F` zooms all the way out, useful for locating a part on a very large or sprawling layout.

A faint **dot grid** marks the 0.1 in hole pitch under everything you place; it coarsens at low zoom and disappears entirely once you're zoomed out too far for individual holes to matter. Zoom and pan position are remembered between sessions.

Breadboards are strips

A real solderless breadboard is not one molded part — it's a centre **pin-board** with one or two **power-rail** strips dovetailed onto its top and bottom edges. Chip Hippo models this literally: each strip is its own

object on the desk, and what looks like "a breadboard" is a small **kit** of strips placed together in one action.

The palette offers three assembled kits:

- **Full 830** — a full-length pin-board with a rail strip above and below.
- **Half 400** — the same layout at half length.
- **Tiny 170** — a bare pin-board only; the real 170-point part ships with no rails at all, so this kit is a single strip.

On a pin-board, rows run top to bottom as `j i h g f`, then the **trench**, then `e d c b a`. The trench is the gap down the centre that isolates the top half of each column from the bottom half electrically — a DIP chip **straddles the trench**, with one row of pins in `f` and the other in `e`, exactly as it would seat on a real board. A power-rail strip carries both a `+` and a `–` line, each one continuous connection along its whole length, independent of every other strip.

See Chips & Components for how chips and discretes seat into the grid, and Power & Clock Sources for feeding a rail from a PSU.

Kits vs. loose strips

Below the assembled kits, the palette also offers the individual strips on their own — a bare **Full pin-board**, a bare **Half pin-board**, a spare **Full power rail**, and a spare **Half power rail**. Reach for these when a board shipped without enough rails, when you want a rail somewhere a kit wouldn't put one, or when you're building up a custom layout strip by strip. Placing, ghosting, and overlap checking all work identically whether you're dropping a whole kit or a single loose strip.

Snapping & mating

Drop a strip within a couple of tenths of an inch of another strip it can dovetail with — matching width when stacked, matching height side by side, meeting flush with no gap — and it snaps into place automatically. This is the same **magnetic pull** whether you're dragging a placed strip or holding a fresh one from the palette; a whole kit snaps as one piece, and the pull only ever engages when the snapped position is still legal (it will never pull a strip into an illegal overlap).

Two strips that end up flush **mate**: they join a shared **group** and from then on drag together as one rigid unit, just like a real breadboard's rails stay attached to its pin-board when you nudge it on the bench. A kit arrives pre-grouped; anything you drop flush against an existing board joins its group (or merges two groups into one, if it bridges a gap). When several strips are mated, clicking any one of them highlights the whole set — the outline traces the union of the group's strips, so flush neighbours read as one board rather than showing a seam between them.

Groups & breaking a snap

Grabbing a mated strip normally drags the **whole group** together. To pull just part of a group apart, hold a modifier while you start the drag:

- **Option-drag** — moves the **reachable run forward** from the strip you grabbed: itself plus every strip mated to it through a below/right edge.
- **Option+Shift-drag** — moves the run **backward** instead, following above/left edges.

Either way, only strips still reachable through the group come along; a strip that merely happens to sit flush elsewhere in the layout is left behind. When you drop a torn-off run, both the piece you moved and what's left of the original group are re-evaluated — each is split into fresh groups based on what's still actually mated within it, so the two halves never end up sharing a group id after the break.

Rotating a rail

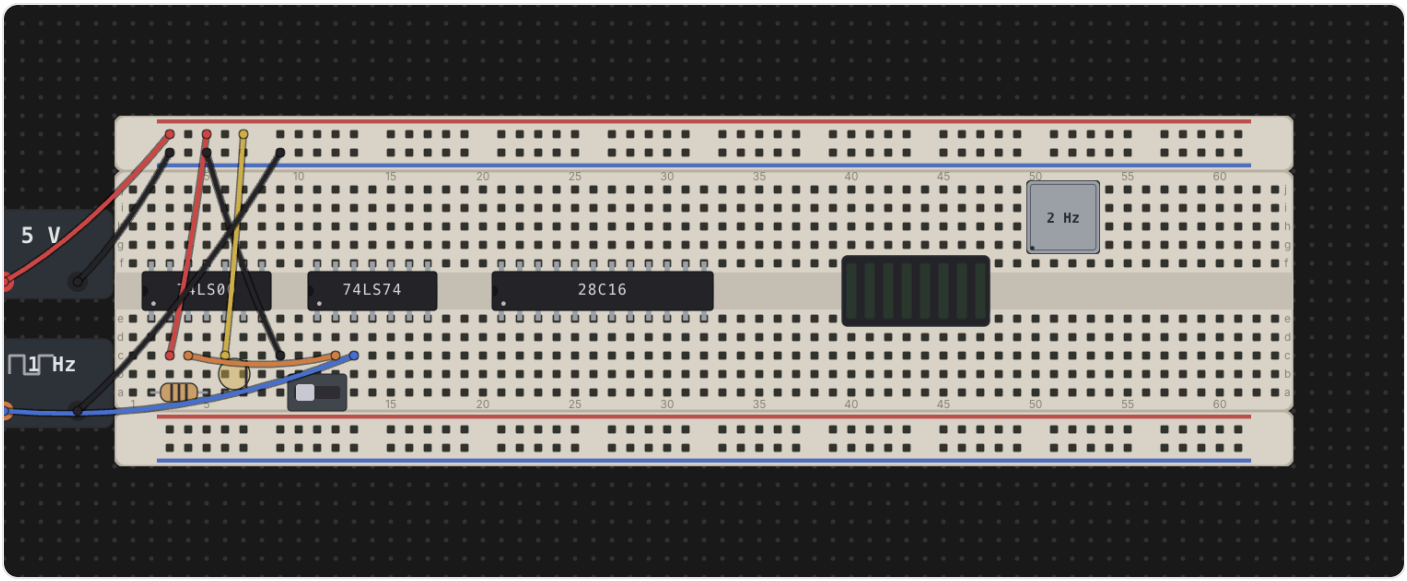
Power rails can rotate; pin-boards cannot. A pin-board's trench (and every DIP seated across it) only makes sense in one orientation, so it's locked at 0°. A rail strip is just two parallel lines of holes, so it reads the same standing on end — turned 90° it becomes a vertical **signal bus** you can run alongside a board and tap into at any point along its length.

Press `R` while a rail is in hand (mid-placement, before you click it down) to cycle its rotation through 0°/90°/180°/270°. Once a strip is placed its angle is fixed; to change it, pick it up again. Rotating doesn't change anything electrically — a rail is one continuous node however it's turned — it only changes which way the strip's footprint runs on the desk.

Next: Chips & Components for populating a board, or Wiring, Nets & Buses for connecting everything up.

Chips & Components

Everything that isn't a breadboard strip lives in the **Parts** palette on the left: 74xx logic chips, memory chips, switches, LEDs, displays, resistors, oscillators, and power/clock bricks. This page covers finding a part, seating it on a board, and the (surprisingly varied) ways different parts rotate and flip once they're down.



The parts palette

The palette opens with every section collapsed, grouped by function:

- **CHIPS** — every 74xx logic family, folder-grouped, plus a separate **Memory** group for ROM/RAM chips.
- **COMPONENTS** — **Switches**, **Resistors**, **LEDs**, **Displays**, **Oscillators**, and **Power**, in that shelf order.

Type in the **Filter parts...** box to search by id, title, or description — matching, it forces every group open so results aren't hidden behind a collapsed folder. A chip whose behavior is wired into the simulator carries a small **sim** badge next to its name (every chip in the catalog currently does). Click any entry to arm placement — a ghost of the part follows your pointer until you click it down on a board, or press `Esc` to cancel.

Placing a DIP chip

A DIP chip always straddles the trench: half its pins seat in row **e**, the other half in row **f**, running the standard counterclockwise DIP numbering — pin 1 at the anchor hole in row e, pins continuing left to

right along e, then wrapping back right to left along f. The **notch** end of the chip (or the dot beside pin 1) marks pin 1 and always faces left. Move the ghost over a pin-board and it snaps to the nearest legal seat; it turns red if the seat is already occupied or falls off the edge of the board.

Because a chip's footprint always occupies rows e/f no matter how it's turned, placing one is a matter of picking the column — there's no click-to-rotate step while placing a chip the way there is for a rail or a resistor; instead, rotation happens afterward (see below).

Placing a discrete

Most discretes — slide switches, push buttons, toggle buttons, LEDs (in their default horizontal form), single-digit 7/8-segment displays, and LED bars (bar8) — are **linear**: they seat along a run of adjacent holes in any single grid row (any of `a` – `j`), not just rows e/f. Drop one anywhere its footprint fits and every free hole underneath it is available.

A few parts don't fit that linear model:

- **bar8iso** — the isolated 8-segment LED bar — is packaged as a 16-pin DIP, so it straddles the trench exactly like a chip: anodes A1–A8 in row e, cathodes K1–K8 in row f.
- **DIP switch banks** (**sw-dip1**, **sw-dip2**, **sw-dip4**, **sw-dip8**) are likewise DIP-packaged (2/4/8/16 pins for 1/2/4/8 switch positions), straddling the trench the same way: each position's two facing pins — one in row e, one in row f — are its own independent SPST switch.
- **Oscillator cans** (**osc-full**, **osc-half**) are rigid four-cornered shapes rather than a line of pins — a full can is 7 holes by 4, a half can 4 holes square, with legs only at the four corners. A can can seat anywhere on the grid, including straddling the trench, since its shape (not a row) determines its footprint.

Rotating & flipping

Rotation behavior is **not one rule for every part** — it depends on what kind of part it is. This is the part worth reading carefully.

Chips (`R`, mid-drag or while selected). A DIP chip's footprint maps onto itself when flipped — same two rows, same columns — so flipping only reverses which physical pin sits where; the chip never has to move. Select a placed chip and press `R` to flip it 180° in place, or press `R` while mid-drag to flip it before you drop it. Either way its pin-assignments window updates to show the new numbering.

bar8iso and DIP switch banks (`R` while selected). The isolated LED bar and every DIP switch bank are DIP-packaged, so they flip exactly like a chip: `R` turns the part 180° in place, the same holes, only the pin numbering per position reverses (a switch bank's own position states don't move — position 1 is

still position 1, just wired to the opposite pins now). Neither has the turn-in-hand behavior of a plain LED — treat them as chips for rotation purposes.

Resistor and LED (`R`, both while placing and once placed). These two-lead parts start in a horizontal **footprint** form (pin 1 and pin 2 a fixed span apart along one row). Press `R` while the ghost is armed to turn it into a vertical **two-free-ends** form instead: pin 1 stays at the anchor hole, and pin 2 becomes a free lead that can land on any other free hole — including a hole on a different strip entirely, such as reaching up to a power rail. Each `R` press while placing steps the ghost a further quarter turn, cycling through all four compass directions before repeating. Once the part is placed, select it and press `R` to rotate it 90° at a time — pin 1 stays put and pin 2's lead swings around it, hunting for the next free hole to land in.

Oscillator cans (`R`, but the exact step differs by state). A can spins around its own centre, not around one pin, and the step size changes depending on whether you're still placing it:

- **While placing**, `R` steps the ghost a full 90° quarter-turn each press, so you can hunt through every orientation for one that fits.
- **Once seated and selected**, `R` behaves differently for the two sizes: the square **osc-half** can still steps 90° at a time, but the rectangular **osc-full** can jumps straight from 0° to 180° (and 90° to 270°) — because a non-square footprint only has two genuinely distinct orientations once it's down (rotating it a further 90° would just retrace the same two footprints it already swept through while placing).

LED polarity (`F`, while placing only). Independent of rotation, press `F` while an LED's placement ghost is armed to flip which lead is the anode and which is the cathode, before you click it down.

Occupancy — one hole, one lead

Every hole on a breadboard — and every terminal on a power/clock brick — holds **at most one lead**, whether that's a chip pin or a wire end. Placing a part checks every one of its derived pin positions against every other part and every wire already on the desk; if any pin would land on an occupied hole, or off the edge of a board entirely, the ghost turns red and the drop is refused. This is the same rule a wire's endpoints follow (see Wiring, Nets & Buses) — chips, discretes, and wires all compete for the same holes, with no separate bookkeeping for any of them.

A rotated resistor or LED's free lead is the one case where a pin can legally resolve to **nothing** — if you later move or delete the strip under that lead, the part stays exactly where it is and that leg simply floats, unconnected, just as it would on a real bench.

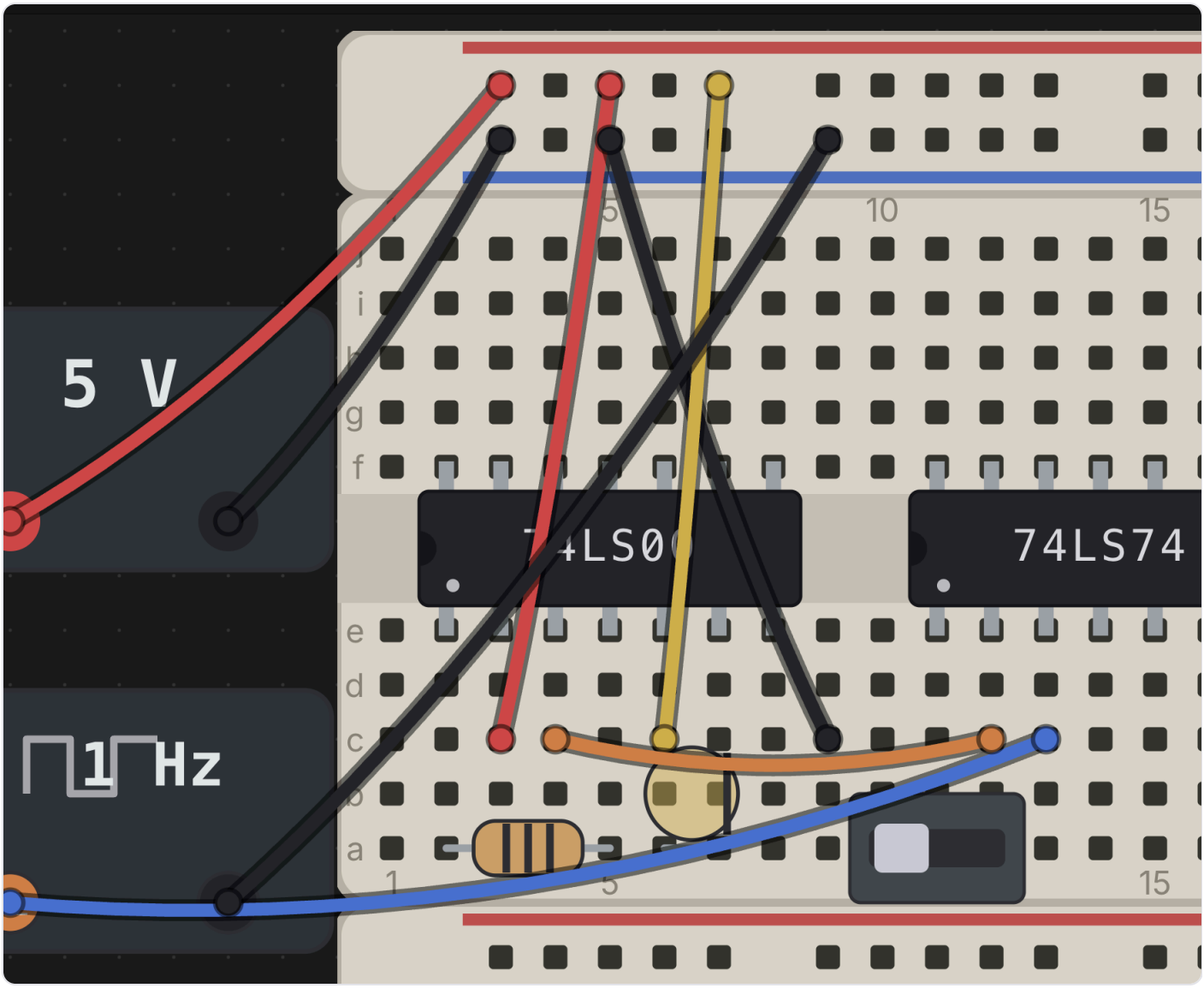
The pin-assignments window

Right-click any placed part — chip, discrete, or brick — and choose **Pin Assignment**, at the top of its context menu, to open its floating pin-assignments window: a diagram of every pin/terminal and, for most chips, a cropped datasheet excerpt below it. A real chip's diagram stays fixed at its canonical layout no matter how you've flipped it on the desk (it matches the physical part, not the placement); a DIP-packaged discrete — `bar8iso` or any DIP switch bank — is the exception: its diagram reflects its current `R` flip, since it has no real notch of its own. See The 74xx Chip Library for the full detail on what the window shows and how it sources its datasheet crops.

See also: The Desk & Breadboards for how boards and strips work, and Wiring, Nets & Buses for connecting components together once they're placed.

Wiring, Nets & Buses

Wires are how you connect the holes, rails, and terminals you've already populated with boards and parts into a working circuit. Chip Hippo's wire tool is click-click (no drag-to-draw), every wire remembers the two *addresses* it connects rather than raw screen positions, and a bus lets you lay a whole marching group of wires — a data or address bus — as one gesture. This page covers the wire tool, colors, cross-board wiring, re-routing and deleting a wire, addresses, and buses.



The wire tool

Press `W` (or click **Wire** in the toolbar) to arm the wire tool. With the tool armed:

1. Click a free hole or terminal to anchor the first end — a rubber-band preview then tracks your cursor.

2. Click a second free point to drop the wire between them.

The tool stays armed after a wire is committed, so you can lay a whole chain of wires without re-arming — each new wire starts fresh, so it does **not** automatically continue from the last endpoint; just click a new starting hole. A hover ring shows the point under your cursor and turns red the moment it's over an illegal target (already occupied, or the same point you started from) — the rubber band tints red too, so a bad landing is obvious before you click.

Press `Esc` to cancel a pending (anchored-but-not-yet-dropped) wire without leaving the tool, or `Esc`/click **Wire** again to disarm it entirely. `M` also disarms whichever of the wire, bus, or probe tools is currently armed — one key to "put the tools away." The wire tool (like the bus and placement tools) is unavailable while a simulation is running; editing is locked until you press **Stop**.

Wire colors

Chip Hippo ships eight wire colors: **red, black, blue, green, yellow, orange, white,** and **purple**. The active color is shown as a swatch dot on the toolbar's **Wire** button — click it to open the color palette, or, while the wire tool is armed, press a digit `1`–`8` to jump straight to that color (1 = red, 2 = black, and so on through the list above). The color you pick **stays pinned** between wires — laying a chain of jumpers keeps whatever color you last chose, so pick red for a supply run and black for ground and they'll stay that way until you deliberately switch.

You can also recolor a wire after the fact: right-click it and choose a new color from the context menu (which also offers **Remove wire**).

Cross-board wires

A wire's endpoints don't care which board or brick they land on — you can run a wire from one breadboard to another, from a board to a PSU terminal, or between two separate strips, exactly as freely as within one board. When you drag a board that a wire is attached to, the wire's other end stays put and the whole wire stretches and re-sags live, riding the move; nothing needs to be re-laid. Deleting a board cascades: any wire with an end seated on that board is removed along with it.

Re-routing & deleting a wire

Grab a wire two different ways:

- **Its end cap** — drag just one endpoint to a new hole or terminal, re-routing that end while the other stays put. A hover ring and red/legal tint on the dragged end work exactly like placing a fresh wire; release over an illegal point and the end snaps back to where it started.

- **Its body** — drag anywhere along the wire itself to translate the whole wire rigidly, keeping its length and orientation and just sliding both ends together onto a new pair of holes. If either landing point isn't free, the wire snaps back on release.

To delete a wire, click it to select it, then press `Delete` or `Backspace` (the same shortcut removes whatever's currently selected — a part, a board, or a wire).

Addresses

Wires never store pixel positions — they store **addresses**, the one cross-module currency for anything wireable. An address is `<ownerId>.<point>`:

- `bb1.a12` — a grid hole (board id `bb1`, hole `a12`).
- `bb2.+7` — hole 7 on a rail strip's `+` rail.
- `psu1.+` — the `+` terminal of a PSU brick.

A wire's `from` / `to` are just two of these addresses, plus a `color`. One hole or terminal holds at most one lead — one pin or one wire end — so the occupancy check that keeps the wire tool's hover ring legal/illegal is the same one that governs seating a chip or a discrete. Because everything resolves through addresses rather than coordinates, a wire keeps working correctly no matter how the boards around it get rearranged.

Buses

For a wide, tidy bundle — a data or address bus — press `B` (or click **Bus** in the toolbar) to arm the bus tool instead of laying eight wires by hand. It's click-click like the wire tool: anchor a start hole, then click a second point to lay the whole run in one go, rendered as a single fat band rather than a fan of individual wires.

Before you click the second point, set what you're laying:

- **Name** — type a bus name like `D[7:0]` or `A[0:15]` in the toolbar; the tool parses the bit range from it. Laying a run onto a **chip pin group** (a catalog-defined bus of pins, like a chip's data lines) instead fans the bus directly onto those pins in bit order.
- **Width** — while the bus tool is armed, press `1` for an 8-bit bus (`D[7:0]`) or `2` for a 16-bit bus (`D[15:0]`), the same two presets the toolbar's width menu offers.

A bus is metadata layered over the individual wires it lays — right-click a placed bus to **Rename**, **Recolor** (which recolors every member wire too), **Un-bundle** (keep the wires, drop the bus grouping), or **Delete bus

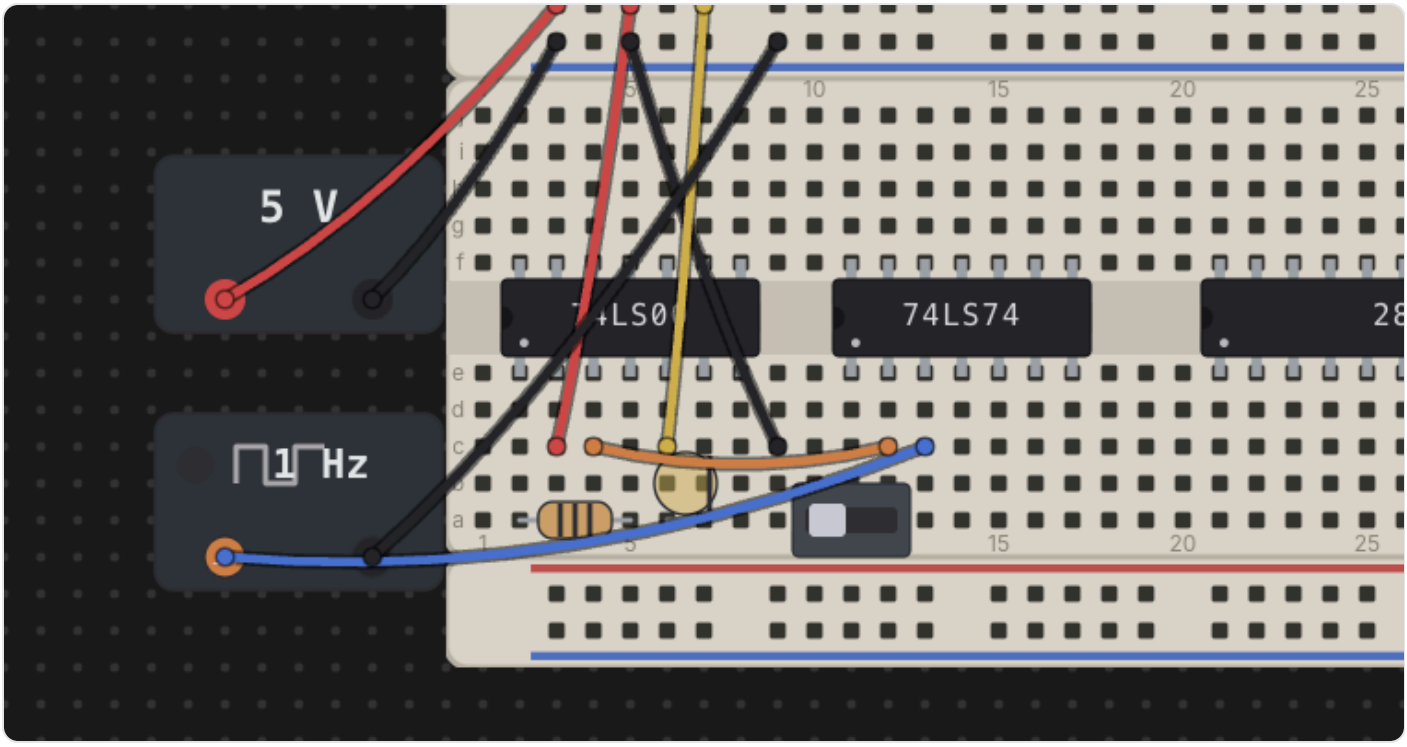
- wires**. Drag a bus by its band to move every member wire together.

See also

- Chips & Components — placing the parts a wire connects.
- Power & Clock Sources — PSU and clock brick terminals, which wires reach exactly like board holes.
- Probing & Net Names — inspecting the nets your wires and buses form, and naming them.

Power & Clock Sources

Every circuit needs somewhere to draw power from and, for sequential parts, a signal to step them along. Chip Hippo gives you two kinds of desk-level **bricks** for this — a **power supply (PSU)** and a **clock source** — neither of which seats on a breadboard. They sit loose on the desk with their own addressable terminals, and you wire those terminals into a board's rails (or straight to a chip's pins) just like any other wire run.



PSU bricks

Add a **Power supply** from the parts palette (**Power** group) and drop it anywhere on the desk — it isn't tied to a board. It draws as a small body with a voltage badge and two terminal pads: a red `+` and a black `−`. Those pads are addressable wire endpoints, `psu1.+` and `psu1.−` for the first PSU you place, exactly like a breadboard hole — click-click a wire from each terminal into a power rail (or directly to a chip's VCC/GND pins) to energize a circuit.

A PSU has no on/off switch of its own — it's always "live" the moment the simulation is running; what matters is which voltage it's set to and what it's wired into.

Choosing a voltage — and the 12 V damage rule

Right-click a PSU brick and choose **Properties...** to pick its voltage: **3 V**, **5 V**, or **12 V**. It's a live setting — the dropdown stays available and applies immediately even while the simulation is running, so you can change voltage on the fly and watch the effect.

- **5 V** — normal operation. A chip whose VCC net carries a 5 V supply and whose GND net is properly grounded runs exactly to its datasheet behavior.
- **3 V** — underpowered. The chip is inert — every output floats/reads as if disconnected — but nothing is harmed. Useful for demonstrating what an underpowered chip looks like without any risk.
- **12 V** — **damage**. This is Chip Hippo's "magic smoke" rule: any chip (or oscillator can, which is powered the same way) whose VCC net sees 12 V is immediately marked **damaged** and goes permanently inert, independent of anything else on the net. Damage is real bookkeeping, persisted with the circuit — it doesn't clear on its own, and it doesn't clear on Stop.

A chip can also come up **reversed** — a PSU `-` on its VCC pin's net at the same time as a PSU `+` on its GND pin's net — which is reported separately from plain unpowered/underpowered, since it specifically means the supply leads are swapped.

Live chip health (powered / underpowered / reversed / damaged) shows as a badge on each chip while running — see Running a Simulation for how those badges and the rest of the settle model work. This page only covers what puts a chip into each state.

Replacing a damaged part

A damaged chip or oscillator can stays damaged and inert until you swap it out. There's no in-place repair — delete it (**Delete Component** on its context menu) and place a fresh one from the palette, then rewire it. 12 V is meant to sting a little, the same way it would on a real bench.

Clock sources

Add a **Clock source** from the palette (**Power** group) for a free-running or manually stepped square wave to drive a sequential chip's clock pin. Like a PSU, a clock brick is desk-level — it doesn't seat on a board — and exposes two addressable terminals: **`out`** and **`gnd`** (`clk1.out` / `clk1.gnd`). Wire `out` to a chip's clock input and `gnd` to your circuit's ground.

Right-click a clock brick and choose **Properties...** to set its rate:

- **1 / 2 / 5 / 10 Hz** — free-running. Once the simulation is running, the brick toggles its `out` level on its own at the chosen rate; a small lamp on the body lights while the output is HIGH.

- **Manual** — click-to-toggle. No timer runs it; instead, while the simulation is running, clicking the brick's body flips `out` from LOW to HIGH (or back) once per click — handy for single-stepping a counter or flip-flop by hand and watching each edge land.

The rate dropdown, like the PSU's voltage one, is a live setting — it applies immediately and stays available while running, so you can retune a clock's speed mid-simulation.

An **oscillator can** (a discrete part that seats directly on a board rather than as a desk brick) behaves the same electrically — it's a free-running square-wave source powered like a chip, with its own rate field in its Properties dialog — but it only ever free-runs; a real crystal has no click-to-toggle pin, so it has no manual mode.

The transport drives the edges

Free-running clocks (and oscillator cans) don't tick on their own outside a simulation — their edges are driven by the **Run/Pause/Step/speed transport** in the header, described fully in Running a Simulation. Briefly: **Run** (`Space`) starts every free-running clock at its configured rate; **Pause** freezes them in place without stopping the simulation; **Step** advances every free-running clock by exactly one half-period and re-settles the circuit, useful for watching a sequential chain edge by edge; and the **speed** control scales every free-running clock's rate together (it has no effect on a manual clock, which only ever moves on a click). A manual clock only responds to clicks while the simulation is actually running — stopped, its body is inert like everything else on the desk.

See Wiring, Nets & Buses for how to route power-rail and clock wiring generally, and Running a Simulation for how power state and clock edges feed into the settle model and live views.

The 74xx Chip Library

The parts palette's **CHIPS** folder holds a broad shelf of 74xx-family DIP logic — everything from a single quad NAND gate up to octal shift registers and 4-bit counters — every one with a datasheet-accurate pinout and, where the simulator supports it, real behavior you can wire up and run. A separate **Memory** group sits alongside it for address-indexed ROM/RAM parts, which get their own dedicated page. This page is a tour of what's on the shelf and how to read a chip's pin-assignments window once you've placed one.

Combinational gates

The basic gate families — the classic 7400-series building blocks:

Part	Description
`74LS00`	Quad 2-input NAND
`74LS02`	Quad 2-input NOR
`74LS08`	Quad 2-input AND
`74LS10`	Triple 3-input NAND
`74LS11`	Triple 3-input AND
`74LS20`	Dual 4-input NAND
`74LS27`	Triple 3-input NOR
`74LS30`	8-input NAND
`74LS32`	Quad 2-input OR
`74LS86`	Quad 2-input XOR

Alongside them, the inverter and buffer/bus-driver parts:

Part	Description
`74LS04`	Hex inverter
`74LS05`	Hex inverter, open-collector
`74LS14`	Hex Schmitt-trigger inverter
`74LS125`	Quad tri-state buffer, active-low enable per gate
`74LS240`	Octal inverting tri-state buffer/line driver
`74LS244`	Octal (non-inverting) tri-state buffer/line driver
`74LS245`	Octal bidirectional bus transceiver — the one part in the catalog with true bidirectional pins

---

Sequential & MSI parts

Everything with internal state, plus the mid-scale-integration decoders and multiplexers that build address/data logic around them.

Flip-flops & latches

Part	Description
`74LS73`	Dual JK flip-flop, clear
`74LS74`	Dual D flip-flop, preset & clear
`74LS76`	Dual JK flip-flop, preset & clear
`74LS107`	Dual JK flip-flop, clear
`74LS112`	Dual JK flip-flop, preset & clear
`74LS174`	Hex D flip-flop
`74LS175`	Quad D flip-flop
`74LS173`	4-bit D register, tri-state
`74LS273`	Octal D flip-flop, clear
`74LS75`	4-bit bistable (transparent) latch
`74LS279`	Quad $\overline{SR}$ latch
`74LS259`	8-bit addressable latch
`74LS533` / `74LS573`	Octal transparent latch, tri-state (inverting / non-inverting)

The `74LS73`, `74LS75`, and `74LS76` reproduce their datasheet's **non-standard power-pin placement** — real parts don't always put VCC and GND on the package corners, and neither do these.

Counters & shift registers

Part	Description
`74LS90`	Decade (÷10) ripple counter
`74LS161`	Synchronous 4-bit binary counter
`74LS169`	Synchronous 4-bit up/down counter
`74LS193`	Synchronous up/down 4-bit counter
`74LS164`	8-bit serial-in, parallel-out shift register
`74LS165`	8-bit parallel-in, serial-out shift register
`74LS595`	8-bit shift register with output storage latch

Decoders & multiplexers

Part	Description
`74LS138`	3-to-8 line decoder
`74LS139`	Dual 2-to-4 line decoder
`74LS151`	8-to-1 line multiplexer
`74LS153`	Dual 4-to-1 multiplexer
`74LS157`	Quad 2-to-1 selector
`74LS257`	Quad 2-to-1 selector, tri-state

Arithmetic, comparison & encoding

Part	Description
`74LS47`	BCD-to-7-segment decoder/driver
`74LS85`	4-bit magnitude comparator
`74LS148`	8-to-3 priority encoder
`74LS283`	4-bit binary full adder

Memory chips

The **Memory** group carries the address-indexed parts: a couple of generic teaching ROM/SRAM chips plus real-shaped EEPROM/EPROM/SRAM parts on wider DIP packages. Reads and writes are wired up the same way as every other chip in the catalog, but a non-volatile chip's contents live in a real file on disk and are programmed through a dedicated in-app tool rather than by the circuit itself. See Memory Chips & the Inspector for the full story — file-backing, the external programmer, and the hex/ASCII inspector.

The sim-ready badge

Every chip in the palette that has real behavior wired up — its truth table or state machine, not just its pinout — carries a small **sim** badge next to its name. Hover it for a reminder: *"Behavior defined — ready for the simulator."* In practice this covers the whole 74xx shelf on this page; you'll only ever see a chip without the badge if a future part ships with a pinout but no behavior yet.

The pin-assignments window

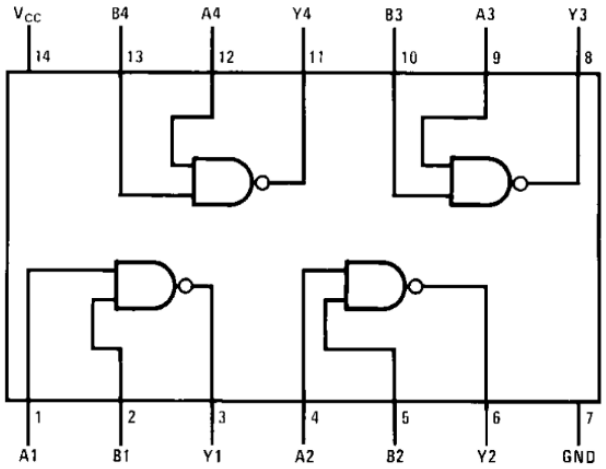
Right-click **any** chip — or a package-footprint discrete like the `bar8iso` LED bar, which seats and rotates exactly like a DIP chip even though it isn't one — and choose **Pin Assignment**, at the top of its context menu, to open its **pin-assignments window**: a small, floating window separate from the main desk, showing the physical DIP layout with the notch at the top, pin 1 at the top-left, and pin numbers wrapping down the left side and back up the right to the highest pin at the top-right, exactly as printed on the part.

74LS00 · Quad 2-input NAND

DIP-14 · pin assignments

Datasheet — internal diagram & function table

Connection Diagram



Function Table

$Y = \overline{AB}$

Inputs		Output
A	B	Y
L	L	H
L	H	H
H	L	H
H	H	L

H = HIGH Logic Level
L = LOW Logic Level

For a real chip this layout is always the **canonical, fixed arrangement** — it matches the physical part regardless of how you've flipped it on the desk, because a real chip's pin-1 dot is a physical feature of the package, not something rotation changes. `bar8iso` is the one exception: it has no real notch to key off, so its pin-assignments window reflects its **current** `R` **flip** on the desk — rotate it and the corners in the dialog swap to match.

Below the pin map, chips that have one show the manufacturer's **datasheet crop** — a cropped image of the connection diagram or function table pulled straight from the source datasheet PDF. If a chip has no crop on file, that part of the window simply isn't there — the pin map alone is still complete and accurate.

When **Settings → Data Sheets** points at a local folder containing that chip's full datasheet PDF, the window also grows a small document button in its top-right corner; clicking it opens the PDF itself in your system's PDF viewer. This is independent of the built-in datasheet crop — you may have one, both, or neither for any given chip.

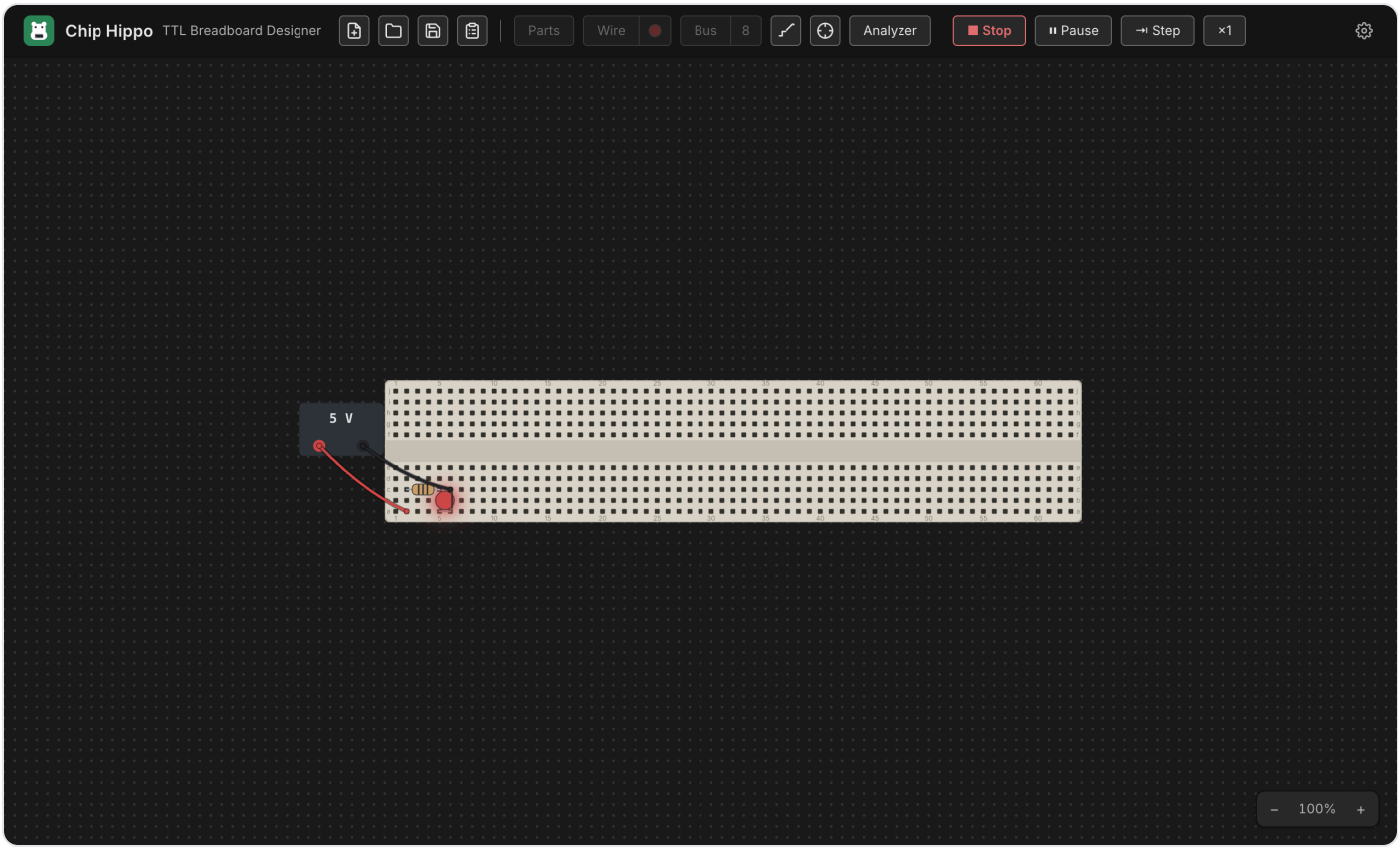
Datasheets

The datasheet crops shown in the pin-assignments window are committed image assets built once from the manufacturer PDFs (`make datasheets`), not fetched or rendered at runtime — they work offline and load instantly. Four parts in the sequential/MSI wave have no matching `74LS*` datasheet on file (`74LS164` , `74LS193` , `74LS27` , `74LS76`) and simply show their pin map with no crop below it. Pointing **Settings → Data Sheets** at your own folder of manufacturer PDFs is a separate, optional feature — it doesn't add or replace the built-in crops, it just adds the "open datasheet PDF" button for any part whose PDF you have on hand.

See also Chips & Components for how chips seat into a breadboard, and Memory Chips & the Inspector for the memory group's file-backing and inspector.

Running a Simulation

Once a circuit is wired up, **Run** hands it to the simulation engine, which traces power from every supply, resolves the electrical state of every net, and settles the circuit — LEDs light, chips report their health, and the probe and logic analyzer come alive. This page covers the transport (Run, Pause, Step, speed), what the signal levels mean, what triggers a re-settle, and what you can still interact with while the circuit runs.



Run & Stop

Press **Run** in the header toolbar — or the shortcut `Space` / `Cmd+R` (`Ctrl+R` on Windows/Linux) — to start the simulation. The button becomes **Stop**, editing locks (you can't move parts, place new ones, or lay wires while running), and the engine cold-starts: it settles the circuit once from scratch and begins driving live views from the result.

Press **Stop** (same shortcut) to freeze the simulation and return to editing. Stopping clears all run-volatile state — net levels, chip health badges, and any sequential state (counter values, shift-register contents, clock phase) reset. The only thing Stop doesn't discard is **12 V chip damage**, which is written into the circuit itself and persists until you replace the damaged chip.

`Space` is disabled while a placement, wire, or bus tool is armed (it might want the key for its own gesture) or while a text field has focus — use `Cmd/Ctrl+R` in that case, or just click **Run**.

Signal levels: H, L, Z, X

Every point in the circuit — a hole, a pin, a wire — settles to one of four levels. The probe tool and the logic analyzer both speak this same vocabulary (see Probing & Net Names):

- `H` — logic high, driven by a supply or a chip output.
- `L` — logic low, driven the same way.
- `Z` — floating: nothing is actively driving this point. Chip Hippo models real 74xx TTL behavior here — **a floating input reads as HIGH**, so an unconnected chip input doesn't just do nothing, it behaves as if tied high.
- `X` — a conflict: two drivers disagree on a net (two chip outputs fighting, or opposing supplies wired together), or the net never settled to a stable value at all (an oscillation — see below).

You don't need to know how the engine computes these to use the app, but recognizing `X` on a probe reading is the fastest way to spot a wiring mistake.

What re-settles the circuit

The circuit isn't a one-shot calculation — anything that could change what a net carries triggers a fresh settle while you're running:

- Clicking a **switch** or **button**.
- Turning a **PSU** on/off or changing its voltage.
- Changing a **clock**'s rate or manual/free-running mode.
- A manual clock click, or (for sequential circuits) a **Step** advance.

Each re-settle starts from the *previous* stable state rather than from scratch — this warm start is what lets a cross-coupled latch or a counter hold its value across small changes instead of resetting every time you touch something.

The transport — Pause, Step & speed

For circuits with a **clock source**, the toolbar grows a small transport cluster next to **Run** the moment you start:

- **Pause** / **Resume** — freezes the free-running clock's edges without stopping the simulation; everything stays lit exactly as it was. Click again (the button relabels to **Resume**) to continue.

- **Step** — advances the circuit by exactly one clock half-period: every free-running clock flips once, and the circuit settles around it. Stepping implies Pause, so you can single-step a counter or shift register edge by edge and watch each bit change. A manually-toggled clock steps only on its own click, not on **Step**.
- **Speed** (`×¼` / `×1` / `×4`) — click to cycle the free-running clock rate up or down. It scales every clock brick on the desk together, so a design with more than one clock keeps their relative timing.

See Power & Clock Sources for placing and configuring a clock brick, and Logic Analyzer & Timing for capturing what these edges actually do to your signals over time.

What lights up while running

Live views render entirely from the settled simulation state — nothing is computed by the views themselves:

- **LEDs** light when their anode net is `H` and their cathode net is `L`. Chip Hippo's LEDs are idealized (no series resistor required), but an LED driven directly between a strong supply and strong ground with nothing current-limiting it shows a distinct **burnt** cue instead of lighting normally — the app's way of telling you that connection would fry a real LED.
- **Chips** show a small health badge the moment they're powered: normal chips show nothing extra, an **underpowered** chip (running at 3 V) gets an amber corner dot, and a chip killed by 12 V shows **damaged** — a red X with a smoke cue. Hover the badge for the exact reason (also unpowered or reversed VCC/GND). A damaged chip stays dead — including through Stop and a fresh Run — until you replace it from its right-click menu.
- **Probe highlights** tint whatever net you hover by its live level. Full detail on reading and naming nets lives in Probing & Net Names — this page won't duplicate it.
- **Clock bricks** carry a small pulse lamp that tracks their own current output level in real time.

Interacting live

Switches, push buttons, and clock bricks stay fully interactive while running — click a **slide switch** to flip it, hold a **push button** to press it, click one position of a **DIP switch bank** to open or close it, or click a **manual clock** to pulse it by hand. Each of these is exactly the kind of input event described above: it doesn't just update its own view, it triggers a fresh settle of the whole circuit, so downstream LEDs and chips react immediately.

Everything that edits the circuit's *topology* — placing, moving, or deleting a part, board, or wire — is locked out until you Stop. Part state (a switch position, a button press, a clock's phase) is not topology, so it stays live the whole time you're running.

Oscillations & conflicts

"The circuit settles" just means: the engine keeps re-evaluating every net and chip, feeding each result back in, until nothing changes anymore. Most circuits reach that fixed point in a handful of passes, faster than you can perceive.

Two things can go wrong:

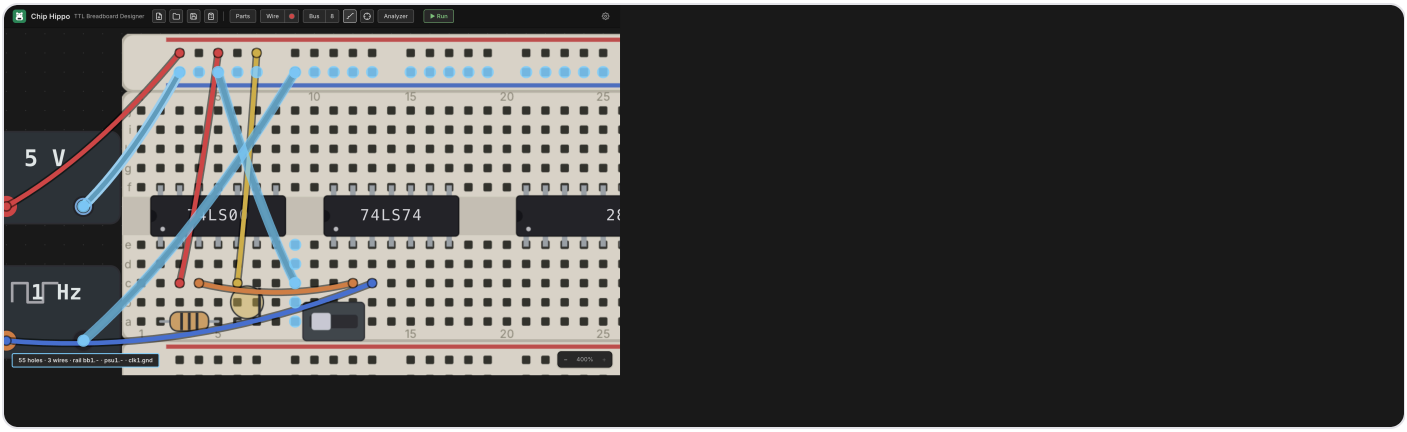
- **A conflict or short** — two chip outputs disagreeing on the same net, or opposing supplies tied together — settles immediately, but to `X`, and the app raises a warning naming the net.
- **An oscillation** — a circuit that never stops changing (an unbuffered ring of inverters, for instance) can't reach a fixed point at all. Chip Hippo detects this — after a bounded number of attempts it gives up, marks the still-changing nets `X`, and flags the oscillation rather than hanging.

Either way, the affected nets read `X` on the probe and in the logic analyzer, which is your cue to go find the wiring mistake causing it.

Next: Power & Clock Sources for supply voltages, the 12 V damage rule, and clock bricks in depth, or Probing & Net Names to read exactly what's happening on any net.

Probing & Net Names

Once a circuit is wired up, the probe is how you ask "what's this actually connected to?" — click any hole, pin, or terminal and Chip Hippo highlights every other point that shares the same electrical net, live, without needing to trace wires with your eyes. This page covers arming the probe, reading its net summary, naming a net so it reads clearly everywhere in the app, dropping free-text annotations on the desk, and sending a net to the logic analyzer.



The probe tool

Press `I` or `P` (either key toggles it — no modifier), or click **Probe** in the toolbar. With the probe armed:

- **Hover** a hole, rail point, PSU/clock terminal, chip pin, or a wire and its whole net lights up — a transient highlight that follows your cursor.
- **Click** a point to **pin** the highlight so it stays put while you move the cursor elsewhere (handy for comparing it against a second net). Click the pinned point again to unpin it.
- `Esc` unpins first, then a second `Esc` disarms the tool; `M` disarms whichever of the wire, bus, or probe tool is currently armed.

Unlike the wire, bus, and placement tools, **the probe stays available while a simulation is running** — editing locks on **Run**, but you can keep probing the live circuit the whole time. See Running a Simulation for what Run/Stop locks and what stays live.

Reading the net summary

A highlighted net draws as a glow: dots on member holes and terminals, rings around member chip pins, and a glow stroke along every member wire — brighter when pinned. Below it, a status readout summarizes the same net in one line, for example:

```
VCC · H · 23 holes · 3 chip pins · 2 wires · rail bb1.t+ · psu1.+
```

The pieces, in order, appear only when relevant:

- The net's **name**, if you've given it one (see below).
- Its current **level** (`H` / `L` / `Z` / `X`) while a simulation is running — gone when stopped, since an unpowered net has no level to report.
- A **connectivity summary** — hole/pin/wire counts, which rails it touches, and any PSU/clock terminals it reaches.

Under the hood, every hole, rail node, wire, active switch/button bridge, and component pin gets partitioned into these nets by a union-find pass over the whole document — the same netlist the simulator resolves each net's level from.

Naming a net

A net's default identity is just its smallest member address (`bb1.a12` , say) — useful internally, meaningless at a glance. Right-click a point while probing to give the net a real name:

- **Name this net…** (or **Rename net…** if it already has one) opens a prompt with quick picks for **VCC**, **GND**, and **CLK**, or type anything you like.
- **Clear name** removes it.

The name binds to the specific point you right-clicked, but resolves to whatever net that point sits on at any given moment — so it survives rewiring around it. If an edit later merges two separately-named nets into one, one name wins deterministically and the other is dropped; nothing crashes, but it's worth a glance if a net you named seems to have picked up its neighbor's label instead.

Naming isn't just cosmetic: a named net is what makes the Schematic View and the Build Guide, Wiring List & BOM readable — `VCC` / `GND` / `CLK` and your own signal names carry through instead of raw hole addresses.

Annotations

For notes that aren't about a net at all — reminders, section headers, "TODO: add a pull-up here" — drop a free-text **annotation** on the desk from the **Annotations** section at the bottom of the parts palette:

- **Label** — a short one-line caption.
- **Note** — a multi-line block.

Picking one arms a placement ghost like a part; click anywhere on the desk to drop it, which opens an inline editor immediately so you can type right away. Drop it **on top of a part** and it **anchors** to that part — the ghost shows the anchored state before you click — so the annotation rides along whenever that part moves. Drop it on open desk and it just sits at that world position.

Double-click an annotation any time to re-edit its text (`Enter` commits a label; a note keeps `Enter` for newlines, so use `⌘Enter` / `Ctrl+Enter` to commit). Drag its body to reposition it, and right-click for **Edit** / **Remove**. Annotations are pure decoration — they carry no electrical meaning and are invisible to the netlist, occupancy, and the simulator.

Sending a net to the analyzer

Probing a net you want to watch over time? Right-click it and choose **Add to analyzer** to pin it as a channel without leaving the probe tool. See Logic Analyzer & Timing for capturing and reading the resulting waveform.

See also

- Wiring, Nets & Buses — laying the wires and buses whose nets you're probing.
- Running a Simulation — what a net's level means and when the probe keeps working while editing locks.
- Logic Analyzer & Timing — capturing a probed net's waveform over time.
- Schematic View — where net names show up in the derived logical diagram.

Memory Chips & the Inspector

Chip Hippo's memory catalog covers both sides of the RAM/ROM divide: address an 8K/32K/128K SRAM for scratch storage that behaves exactly like real silicon (it forgets everything at power-off), or a ROM/EPROM/EEPROM that holds a real byte image between runs — the same image, opened in a floating **inspector**, that you can browse, hand-edit, and program from a file. This page covers what each kind actually stores, how to get data into a ROM, and how the inspector works.

Volatile vs. non-volatile

Every memory chip in the catalog is one of two kinds, and the difference matters more than the pinout:

- **Volatile (SRAM)** — `ram-8k`, `HM62256` (32K×8), `AS6C1024` (128K×8). Contents live only for the current run: they start as random noise every time you press **Run**, whatever the circuit writes stays readable until you **Stop**, and then it's gone. Nothing is ever saved to disk for these parts — there's no image to lose, because there was never a file.
- **Non-volatile (ROM/EPROM/EEPROM)** — `rom-8k`, `28C16` (2K×8 EEPROM), `AM27C1024` (64K×16 EPROM). These chips are backed by a real file on disk in the app's own working folder, so their contents persist across runs and across closing and reopening the document. An unprogrammed chip reads back random noise too — just like an unprogrammed EPROM off the shelf — until you program it. (Exactly which file backs which chip is an internal bookkeeping detail you never need to touch by hand.)

The chip's **blurb** in the parts palette and pin-assignments window always says which kind it is, and its behavior — read timing, chip-enable/output- enable pins, address/data bus width — is otherwise ordinary combinational logic. See The 74xx Chip Library for the full catalog and the pin-assignments window.

Programming a ROM

The circuit itself can never write a non-volatile chip. An EEPROM or EPROM's real write cycle isn't something the simulator drives — Chip Hippo treats every non-volatile part as read-only from the circuit's point of view, the same way you couldn't reprogram a 27C-series EPROM just by wiggling its pins on a breadboard. Any write a chip reports during simulation is silently dropped if the chip is non-volatile.

The only way to put data into a ROM is the **external programmer**:

1. Right-click the chip on the desk and choose **Properties...**.
2. Click **Load image... (program)**.

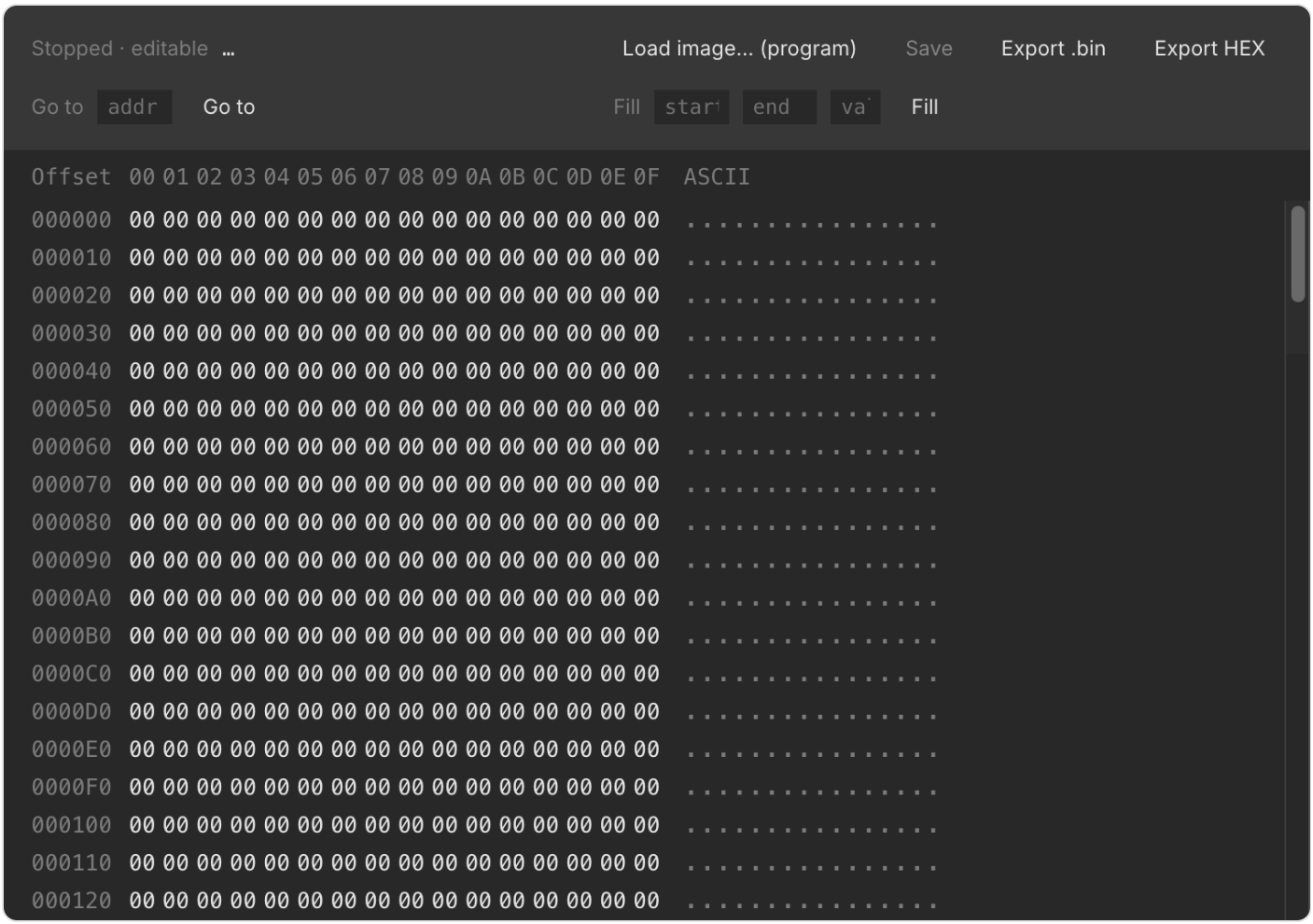
3. Pick a `.bin` (raw bytes) or `.hex` (Intel HEX) file.

The image is copied to the start of the chip's memory. If the file is smaller than the chip, only the start is overwritten and the rest is left alone; if it's larger, it's truncated to fit — either way you get a warning telling you what happened. Once programmed, the chip is flagged **programmed**, which rides undo/redo like any other edit, and the chip keeps its new contents the next time you open the document or press Run.

Programming is only available while editing is unlocked (i.e. the simulation is stopped) — see Running a Simulation.

The memory inspector

Every memory chip — SRAM or ROM alike — has a floating **inspector** window: right-click it on the desk, choose **Properties...**, then **Inspect memory...**. It's a hex/ASCII grid, one row per 16 bytes, with a toolbar for **Go to** (jump to an address), **Fill** (write a byte value across a start/end range), **Save**, and Import/Export.



What you can do depends on whether the simulation is running and what kind of chip it is:

- **Stopped, ROM/EPROM/EEPROM** — the grid is editable. Click a hex or ASCII cell to type a new byte value directly, or select a range and **Fill** it. **Save** writes your edits back to the chip's file and flags it **programmed**, exactly like using the external programmer.
- **Stopped, SRAM** — shows the chip's last contents from before it was stopped (or noise, if it's never been run) as a read-only snapshot; there's nothing to save, since SRAM has no file.
- **Running, any chip** — the grid is a **live, read-only** mirror of the simulator's own byte image. Bytes the circuit writes are tinted as they change, so you can watch a counter or a shift register fill memory in real time. Editing resumes once you stop.

The inspector also shows the chip's backing-file path for a non-volatile part (handy for locating the raw `.bin` outside the app), and stays open and live-updating as long as the desk document does.

Import & export (Intel HEX)

Beyond loading a raw `.bin`, the inspector's toolbar can **Export** the current contents as either a raw `.bin` or an **Intel HEX** `.hex` file — useful for round-tripping through an assembler toolchain or another tool that expects the hex format. The external programmer (above) accepts either extension coming in: a `.hex` file is decoded before it's written to the chip.

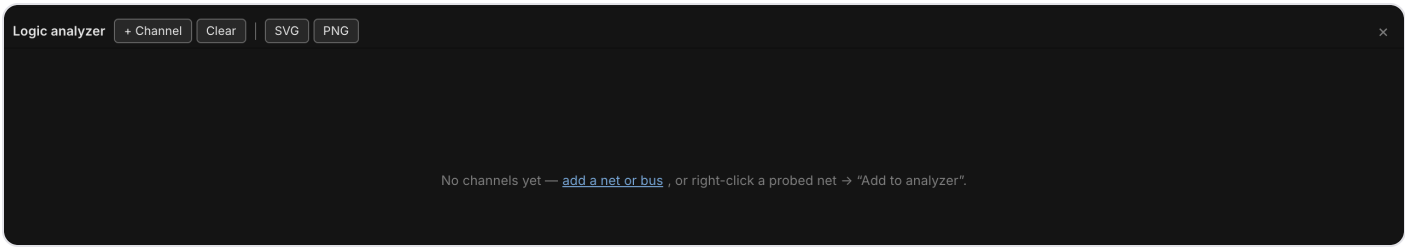
A missing backing file

Because a ROM's contents live in a file separate from the desk document, it's possible for that file to go missing without the document changing — the classic case is deleting the chip and then **undoing** the delete. If Chip Hippo finds a chip flagged **programmed** but its backing file is gone, it recreates the file as fresh noise and warns you that the chip's data was lost, so you know to reload the image rather than trusting stale contents.

See also The 74xx Chip Library for the memory chips in context with the rest of the catalog, and Files, Autosave & Undo for how the desk document itself is saved and versioned.

Logic Analyzer & Timing

The **Analyzer** is a bottom-docked panel that records the live simulation and draws it as a scrolling timing diagram — one lane per signal you've chosen to watch, a step waveform for a single net or a hex value-lane for a bus, with two draggable cursors for measuring the time between two points. It's a passive recorder: it reads the same broadcast the LEDs and chip badges already render from, so opening it, closing it, or adding channels never changes how the circuit behaves.



Opening the analyzer

Click **Analyzer** in the toolbar, or press `Cmd/Ctrl+A`, to dock the panel along the bottom of the window. Click the button again, press the shortcut again, or click the panel's **×** to close it — the panel remembers whether it was open and how tall you left it (drag its top edge to resize, from a compact strip up to half the window).

With no channels added yet, the panel shows a prompt instead of an empty grid: **add a net or bus**, or right-click a probed net and choose **Add to analyzer**.

Adding a channel

There are two ways to add a channel:

- **From the panel** — click **+ Channel** in the header. The menu lists every *named* net (see Probing & Net Names) and every bus in the document that isn't already on a channel; picking one adds it.
- **From the probe tool** — arm the probe (`I`), right-click any net on the desk, and choose **Add to analyzer**. This works on ANY net the probe can reach, named or not — it's the way to watch a signal you haven't bothered to name. Using it also opens the panel automatically if it was closed.

Each channel gets a row in the left-hand gutter: a color dot (cycled from a fixed palette unless the channel has its own color), the net or bus name, its current value, and three small controls — **↑** / **↓** to reorder the channel, and **×** to remove it. Channels are saved with the document (`doc.scopeChannels`), so a schematic reopens with its analyzer setup intact, and adding/removing/reordering a channel rides

the normal undo/redo stack. A channel that loses its target (the net was deleted, the bus was removed) simply reads as undriven rather than disappearing — it comes back to life if the target returns.

Reading a waveform

A single-bit channel (a **net**) draws as a step waveform inside its lane:

- **High** traces along the top of the lane, **Low** along the bottom.
- **Floating (Z)** draws a dashed line through the lane's midline.
- **Unknown/conflict (X)** — including anywhere the net is undriven — fills the region with a diagonal amber hatch instead of a line.

The gutter's value column always shows the channel's current level (or its value at cursor A, once one is placed — see below).

Multi-bit (bus) lanes

A channel bound to a **bus** (see Wiring, Nets & Buses) doesn't draw a two-level waveform — decoding eight or sixteen individual traces would be unreadable. Instead it draws a **value-lane**: a band that necks down to a point at every change and displays the bus's current value as hex (`0x1A` , uppercase) centered in the band, wide enough for the label to fit. The bus is decoded from its member wires' bit positions (as parsed from a name like `D[7:0]`); if even one member is floating or unknown, the whole word is undecodable for that span and the lane hatches just like an unknown net.

Cursors & Δ

Click anywhere in the waveform area to drop **cursor A** at that tick; `Shift` -click to drop **cursor B**. Drag either cursor to slide it to a different tick — the gutter's value column updates to show each channel's value at cursor A while it's placed. With only cursor A down, the header reads `t=<tick>` ; with both down, it reads the delta between them — **Δ N ticks**, plus a millisecond figure when a clock source on the desk gives the analyzer a time base to convert against.

While the simulation is running, the view auto-scrolls to keep the newest tick in sight; scrolling the lanes away from the right edge turns this "follow" behavior off until you scroll back. **Clear** in the header wipes the recorded trace and both cursors without touching the channel list; starting a fresh **Run** does the same automatically.

Export

Two buttons in the header export the diagram exactly as currently rendered — every channel, the full retained trace, and a label column identifying each lane:

- **SVG** — downloads a self-contained `timing.svg` (colors resolved to concrete values, not the app's live CSS variables, so it renders correctly outside Chip Hippo).
- **PNG** — rasterizes that same SVG at 2× scale and downloads it as `timing.png`.

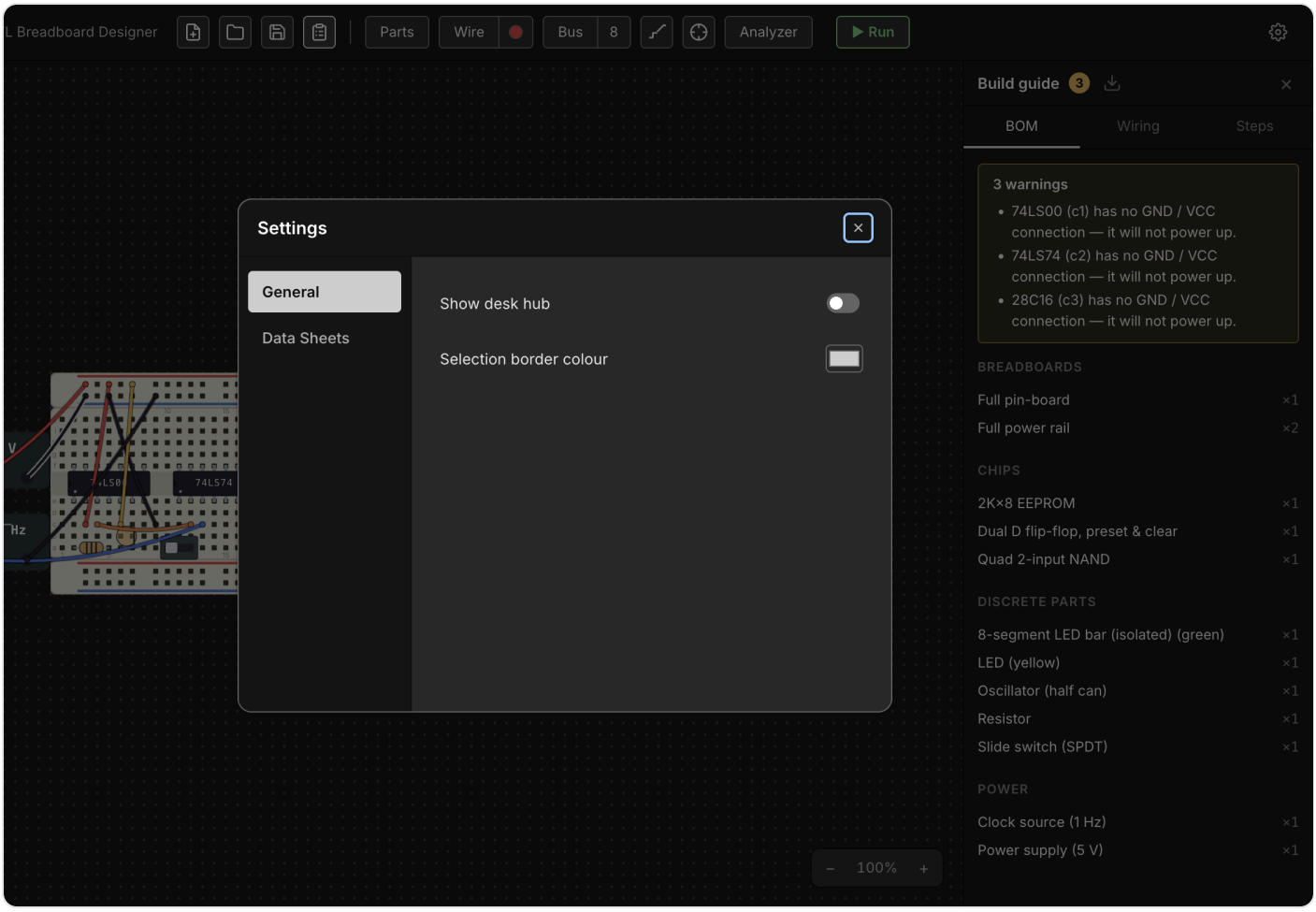
Both are plain browser downloads (no file dialog, no IPC round-trip) and do nothing if there are no channels or nothing has been recorded yet.

See also

- Probing & Net Names — arming the probe, naming a net, and the "Add to analyzer" context-menu entry.
- Wiring, Nets & Buses — laying the buses a bus channel decodes.

Build Guide, Wiring List & BOM

Once a circuit is wired up on the desk, Chip Hippo can turn it into the paperwork you'd actually want on the bench: a bill of materials (what to buy, and how many), a human-addressed wiring list (what connects to what, in words a person can follow with a real breadboard in hand), and an ordered assembly checklist. None of this is stored on the document — it's derived fresh from the live desk every time you open or change it, so it never drifts out of sync with what you've actually built.



Opening the build guide

Click the **Bill Of Materials** clipboard button at the right-hand end of the desk-tools pill, or press `Ctrl+B` / `⌘B`, to open the build guide as a right-docked panel. Clicking it again, or the panel's own **×** close button, hides it — and the button stays lit for as long as the panel is open, however you opened or closed it. The panel's open/closed state is remembered between launches, same as the parts palette. It has three tabs — **BOM**, **Wiring**, and **Steps** — and is read-only, so it stays available even while a simulation is running.

The guide re-derives itself automatically whenever the document changes while it's open (adding/removing a part or wire, renaming a net, flipping a switch) — there's nothing to refresh by hand.

Warnings

Above whichever tab you're on, the guide surfaces anything that looks un-buildable or likely forgotten, with a count badge on the panel's title when there's something to see:

- **Floating leads** — a chip or discrete pin whose lead sits over no hole.
- **Unpowered chips** — a chip whose VCC or GND pin has no connection, so it will never power up.
- **Single-member nets** — a wired net that only reaches one point, which usually means a connection you meant to make and didn't.

The bill of materials

The **BOM** tab lists everything on the desk as counted line items, grouped into four sections (only the non-empty ones show): **Breadboards**, **Chips**, **Discrete parts**, and **Power**. Boards are counted by strip type (a Full 830 kit counts as its constituent rail/pin strips, not as one line), and components are counted by catalog identity — with a few splits that matter for actually buying the right part: LEDs split by color, PSU bricks by voltage, and clock sources by rate. Each line reads as `title ×count`.

The wiring list

The **Wiring** tab is net-centric, not wire-centric: instead of listing raw wire endpoints, it lists every electrically-interesting **net** and what's connected to it. A net only shows up here if you actually connected something to it — either it carries a wire, or it has two or more members — so a freshly-placed 14-pin chip with nothing wired to it doesn't spam the list with fourteen empty rows.

Each net's members are resolved to the friendliest label available, in this priority order:

1. A component pin at the hole — `"74LS00 pin 3 (1Y)"` (part + pin number + datasheet pin name, when the pin has one).
2. A pin sharing the hole's 5-hole node — a bus tap lands *beside* a pin rather than on it, and still resolves to that pin.
3. A PSU or clock terminal — `"Power supply +"`.
4. A power rail — `"+ rail (bb1)"`.
5. The bare hole address, as a last resort.

A net you've named (see Probing & Net Names) leads with that name instead of an anonymous net id, and buses group under a `<bus name> bus` heading with each row labeled by bit — so naming your nets up front, before opening the guide, makes this tab read far more like a real wiring diagram and far less like an address dump. See Wiring, Nets & Buses for how wires, colors, and buses work in the first place.

A net that only reaches one salient member is flagged with a small warning icon right in its row (and rolled into the warnings banner above).

Assembly steps

The **Steps** tab is an ordered checklist for building the circuit from scratch, grouped in the order you'd actually work: **Place the boards**, **Power**, **Seat the chips**, **Add discrete parts**, **Run the signal wires**. Each step has a checkbox — ticking it is a session-only visual aid (nothing is persisted), useful for tracking progress while you build.

A few notable details in how steps are phrased:

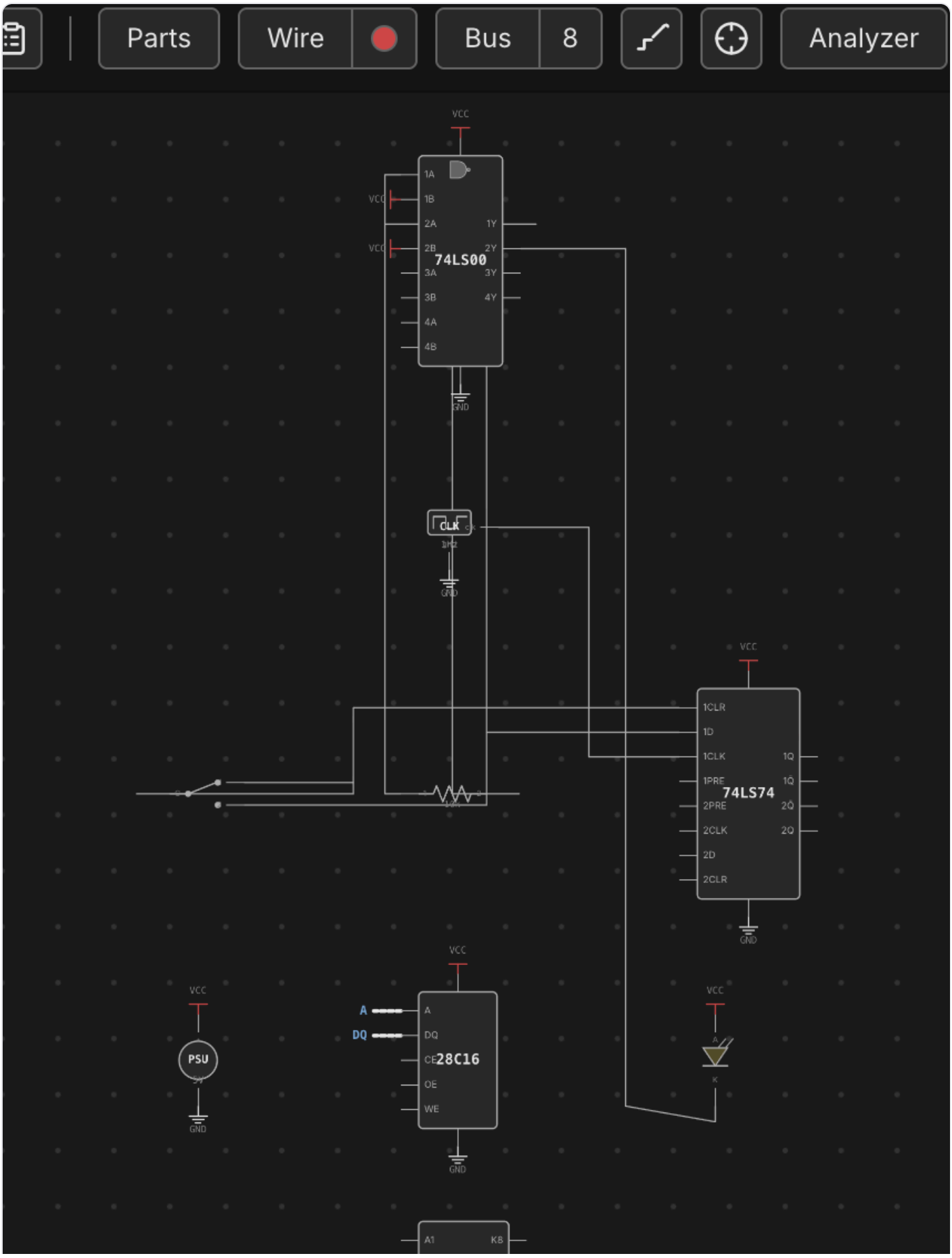
- A grouped breadboard kit (rails + pin strip snapped together) is one step, not one per strip; a loose strip gets its own step.
- Power steps cover PSU/clock bricks (set to their configured voltage/rate) and then any wire that distributes power — a brick terminal or rail hole at either end.
- A chip's step spells out its straddle, e.g. *"straddling e5–f11, pin 1 at bb1.e5"* (plus a flipped note if it's rotated 180°); a linear discrete lists its resolved lead addresses instead.
- Signal wires are grouped last: whole buses first (one step per bus, one detail line per bit), then the remaining signal nets, each with its wires listed as `from → to` using the same friendly local labels as the wiring list.

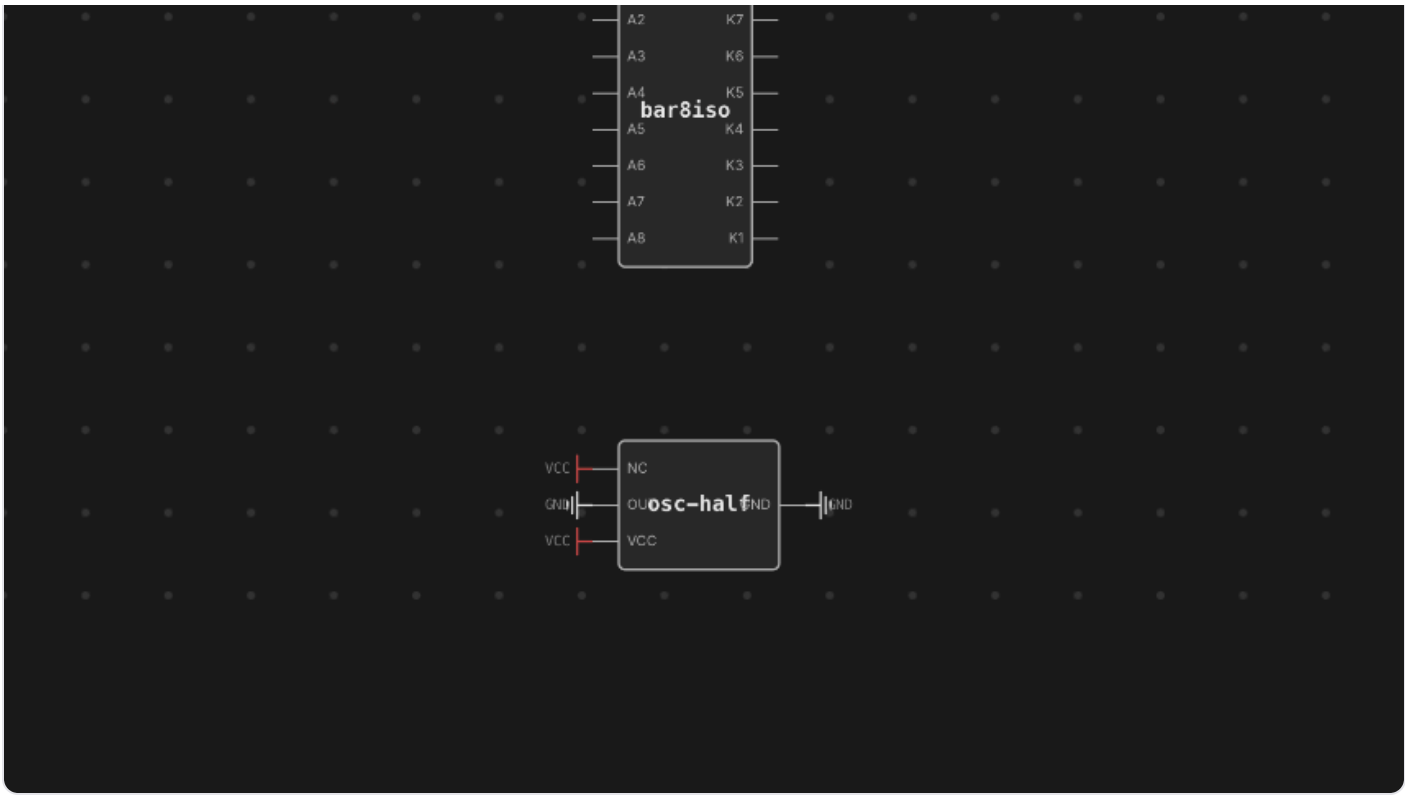
Downloading the BOM

The download icon in the panel header exports the **whole current plan** — BOM, wiring list, and assembly steps together, headings and all — as a Rich Text Format (`.rtf`) document, named `<schematic name>-bom.rtf` after your current schematic file. It's a plain browser download (no save dialog, no main-process IPC involved) generated entirely in the renderer, so it reflects exactly what the panel is showing at the moment you click it. `.rtf` opens in any word processor, which makes it easy to print or hand off as a build sheet alongside the physical parts.

Schematic View

The desk shows your circuit exactly as you built it — a physical breadboard with chips seated in holes and wires running point to point. The **schematic view** shows the same circuit the way an engineer would draw it: chip symbols, named nets, and bus lines, laid out automatically. It's a **projection**, not a second document — the breadboard is still the single source of truth, and nothing you do in the schematic changes a hole, a pin, or a wire on the desk.


Parts
Wire
Bus
8
Analyzer
VCC
1A
1B
2A
1Y
2B
2Y
74LS00
3A
3Y
3B
4Y
4A
4B
GND
CLK
1CLR
1D
1CLK
1Q
1PRE
74LS74
2PRE
2Q
2CLK
2D
2CLR
PSU
A
DQ
28C16
CE
OE
WE
K
A1
KB



Switching views

Press `Tab` to flip between the **Breadboard** and **Schematic** views (not while a text field has focus — typing a `Tab` there behaves normally, e.g. to move between fields). The schematic reuses the desk's own camera, so pan and zoom feel identical in both views and your position is preserved when you switch back. The first time you open the schematic in a session it automatically fits the whole diagram in the viewport; after that, panning and zooming are exactly what you left them.

If the desk has no chips on it yet, the schematic shows a short hint instead of an empty canvas — add a chip on the breadboard and switch back to see it appear.

Chip symbols & routed nets

Every chip is drawn as a labelled box (or, for a plain gate chip, a small distinctive shape — AND/OR/NAND/NOR/XOR/NOT/BUFFER — badged at the top of the box) with named pin stubs instead of physical DIP pins. Discretes get their own recognizable symbols too: an LED's diode triangle, a resistor body, a switch, a push button, a PSU or clock source block. VCC and GND pins don't route across the page like a signal would — each one drops a small power-rail symbol right at the pin, the way a real schematic keeps power off the signal routing.

Everything else is connected by **routed nets**: orthogonal lines that run chip-to-chip along the net they belong to, labeled with the net's name where there's room. A group of pins wired as a bus (see Wiring,

Nets & Buses) collapses into a single **fat bus line** instead of drawing every bit as its own wire — the same visual shorthand a hand-drawn schematic uses for a data or address bus.

Auto-layout & nudging

The schematic lays itself out automatically and deterministically: the same circuit always produces the same diagram, with chips arranged left-to-right roughly along the direction signals flow and edges routed to keep crossings down. You never have to arrange it by hand — but if a particular symbol lands somewhere inconsistent, drag it to a better spot. That drag is stored as a **per-symbol position hint**, not a real coordinate: it only ever affects where that symbol sits on THIS derived diagram, and it has no effect on the component's actual place on the breadboard. Nets simply re-route to follow wherever you drop it.

An **Auto-layout** button in the schematic clears every position hint at once and lets the algorithm re-place everything from scratch — useful after nudging things into a mess, or after a big edit has made your old arrangement stale.

Live simulation tint

Press **Run** and the schematic lights up exactly like the breadboard does: the routed lines tint by their net's live level, an LED symbol lights when its anode reads high and its cathode reads low, and a chip symbol picks up the same health badge (unpowered / underpowered / reversed / damaged) you'd see on its physical package. Both views are driven from the same simulation state, so whichever one you're looking at when you hit Run shows the identical picture — there's nothing to keep in sync by hand. See Running a Simulation for what each level color and health badge means.

The probe is shared

The connectivity **probe** (see Probing & Net Names) is armed and driven from the breadboard, but its highlight is not confined to it: probing a net on the desk highlights that same net in the schematic too — by its stable net id, not by screen position — so you can hover a wire on the physical board and immediately see which chip symbols and bus lines it corresponds to in the logical diagram. The highlight is applied to both views at once, whichever one happens to be visible, so switching to the schematic after probing shows the net already picked out. It's the same underlying net, just drawn two different ways.

See also: The Desk & Breadboards for the physical view this diagram is derived from, and Wiring, Nets & Buses for how wires and buses become the nets the schematic routes.

Files, Autosave & Undo

Chip Hippo separates two things most apps blur together: the circuit you're currently poking at, which is saved for you continuously, and a named schematic file you deliberately save to organize or share a specific design. This page covers both, plus the discard-changes prompt, the title bar's dirty indicator, and undo/redo.

The autosaved working document

Whatever is on the desk right now — every board, chip, wire, and part — lives in a **working document** that Chip Hippo saves automatically as you go. There's nothing to save manually for day-to-day tinkering: place a chip, lay a wire, quit the app, relaunch, and the desk comes back exactly as you left it. This is the same document that opens by default every time you launch Chip Hippo.

Named files

A **named file** (extension `.chiphippo`) is a snapshot of a schematic you've explicitly saved somewhere on disk — useful for organizing a design under a project name, keeping several circuits side by side, or sharing one with someone else. Once you've opened or saved a named file, it becomes the working document: further edits keep autosaving into it, and the title bar shows its filename.

New, Open, Save, Save As

Four commands on the **File** menu cover the whole file lifecycle:

- **New Schematic** (`Cmd/Ctrl+N`) — clears the desk back to an empty working document, no file attached.
- **Open Schematic...** (`Cmd/Ctrl+O`) — shows a native file picker; choose a `.chiphippo` file and it becomes the working document.
- **Save** (`Cmd/Ctrl+S`) — writes the current desk to the file it's already associated with. If there isn't one yet, this falls back to **Save As...**.
- **Save As...** (`Shift+Cmd/Ctrl+S`) — shows a native save dialog; writes the current desk to the chosen path and adopts it as the working file from then on.

The same four actions are also available as buttons in the header toolbar's schematic menu, alongside their keyboard shortcuts.

New and **Open** replace the whole desk, so the window refreshes to load it — a brief flash as everything rebuilds from the new document. Note that this resets any run-volatile simulation state (see below); it

doesn't affect what's saved.

Unsaved changes

The title bar shows the current file's name (or "Untitled" for a fresh document) with a leading dot — `• MyCircuit — Chip Hippo` — whenever the desk has changes that haven't been written to that file yet. Save clears the dot.

Because **New Schematic** and **Open Schematic…** both replace the working document outright, Chip Hippo won't silently throw away unsaved work: if the desk is dirty, it shows a **Discard unsaved changes?** prompt naming the current file and asking you to confirm before proceeding. Choose **Discard** to continue (losing those changes) or **Cancel** to back out and save first.

Undo & redo

`Cmd/Ctrl+Z` undoes the last edit; `Shift+Cmd/Ctrl+Z` redoes it. Undo/redo covers the full editing history of the circuit — placing and moving boards, chips, and discretes; wiring and unwiring; deleting anything; net names — as far back as your session's history allows, and both menu items disable themselves automatically when there's nothing left to undo or redo.

Undo/redo does **not** cover simulation state. While the circuit is running, editing — and with it, recording new undo steps — is locked, so nothing that happens mid-run (sequential chip state, clock phase, a chip taking 12 V damage) ever becomes an undo step of its own. Sequential state and clock phase vanish outright the next time you press **Run**. 12 V damage is the one exception that outlives the run: it's written into the document, so stopping — or even quitting and reopening — doesn't clear it, and there's no `Cmd+Z` back to before it happened. The only way to clear a damaged chip is to delete it and place a fresh one (see Power & Clock Sources) — which, taken while stopped, is a normal edit and undoes/redoes like any other. What undo/redo restores is always the circuit you built, never a moment in its simulated behavior.

Example circuits

Chip Hippo's project repository ships a handful of ready-to-load example circuits as ordinary `.chiphippo` files — currently small W65C02-based breadboard computers built from 74xx glue logic, each paired with a `.hex` ROM image. Open one the same way as any saved file: **File ▸ Open Schematic…**, then load its matching `.hex` into the ROM chip via the memory inspector or the external programmer before pressing **Run**.

See Getting Started for building your first circuit from scratch, and Settings for what else persists between sessions.

Projects & Desktops

A big build eventually needs a bench beside it. You want to work out a clock module, an address decoder, or a memory map *without* doing it in the middle of everything else — and then drop the finished thing into the main design.

That's what a **project** is for. A project is a workspace holding several **desktops**, shown as tabs above the desk. Each desktop is a complete desk of its own — its own boards, chips, wiring, camera position, and undo history — and a design worked out on one can be copied straight onto another.

Every desktop is the same. There is no special "main" desk and no lesser one: they are peers, all with the same menu, and which is the build and which is the bench is entirely up to you. Any of them can be renamed or deleted — the only rule is that a project always keeps at least one.

The tab strip is **always** above the desk, project or not. With no project open it shows the single desk you're on, and the **+** beside it is how you get another.

Starting a project

Just press +. Adding a desktop never asks you anything: the desk you're on becomes the project's first desktop (so nothing you were working on is discarded), the desktop you asked for appears beside it, and you land on it. After that, every + adds one more. A project starts life **Untitled** — the window title says so.

The **Projects** button in the header toolbar has the rest:

- **New Project** — a blank slate: a fresh untitled project holding one new, empty desktop. No dialog. However many desktops you had, a new project always starts as a single one, numbered from `Desktop 1` again.
- **Load Project...** — lists the projects you've saved; pick one to open it.
- **Save Project...** — the one and only place a name is asked for. See below.
- **Add Desktop** — the same thing as the **+** at the end of the tab strip.

Saving a project

Projects ▸ Save Project... asks for a name and keeps the project under it. Every desktop is written to its file first, so "save the project" means all of it, not just the tab list. Once a project is named, the item greys out: there is nothing left to do, because a saved project's desktops and tab list are already kept up to date.

Project names must be unique. If you type one you've already used, Chip Hippo says so and asks for a different one — it never merges two projects into one, and never offers to open the other (that would throw away the work you're saving).

Saved projects are one folder per project, holding a small project file (the tab list) plus one ordinary `.chiphippo` file per desktop. Because a desktop is just a normal schematic file, anything you build on one can be opened outside the project too.

Changing projects

New Project and **Load Project...** both replace what's open, so nothing is allowed to go with it unasked:

- An **untitled** project has to be dealt with whether or not its desktops are saved — it lives in Chip Hippo's one working slot and has no name to come back to it by. You're asked to **save it first** (which names it and writes every desktop), **discard** it, or **cancel** the whole thing.
- A **saved** project with desktops you've changed asks whether to **save all**, **discard**, or **cancel**.
- With no project open, the plain **desk** gets the same courtesy before a project takes the screen from it.

Choose to save and it goes through, then the action you asked for carries on — you're never made to ask twice. Cancel, at that dialog or at the name dialog behind it, and nothing happens at all.

Load Project... shows you the list of saved projects **first**; the question above comes once you've picked one, so backing out of the list costs you nothing.

Working across tabs

Click a tab to put that desktop on the desk. Everything follows the active tab:

- The **toolbar's New, Load and Save** act on that desktop. Save writes the desktop's own file inside the project — no dialog, because a tab already knows where it lives. New empties that desktop; Load reads a `.chiphippo` into it.
- **Undo/redo** is per desktop. Switch away and back, and `Cmd/Ctrl+Z` still undoes that desk's last edit, not the other one's.
- Each desktop remembers **where you were** — its camera position comes back with it.
- The **simulation stops** when you switch. A running circuit belongs to the desk it's running on, so the transport resets rather than following you to another desktop. Any open pin-assignment or memory-inspector windows close for the same reason: they were pointing at chips on the desk you just left.

A dot on a tab (`Desktop 2 •`) means that desktop has changes you haven't saved yet. The window title shows the project — `Untitled` until you name it — and the active desktop.

Managing tabs

Right-click a tab for its menu:

- **Properties...** opens the same dialog every part, board, and wire has, with the same two fields: a **Name** — what the tab reads — and a **Description**, a note on what this bench is for. Hover the tab to see the description; both are saved with the project as you type them.
- **Delete Desktop** removes it, along with its file. Any desktop can go — there's nothing special about the first one — except the **last** one left, because a project with no desktops would have nothing to open. If the desktop has unsaved changes, you're asked whether to **save and delete**, **delete anyway**, or **cancel**.

Desktop numbers only ever count up: deleting `Desktop 2` doesn't make the next one `Desktop 2` again, so a name you wrote in a note keeps meaning the same bench. Rename them to whatever the benches actually are — `CPU`, `Clock`, `Decoder` — through Properties...

Copying a design from one desktop to another

This is the point of the whole thing.

1. On the desktop holding it, **shift-drag a marquee** around the design — the boards, the chips on them, and the wiring. A marquee takes in any board it encloses *completely*, and the selected set is outlined as one block.
2. Press `Cmd/Ctrl+C`. The whole sub-assembly is copied: the boards, everything seated on them (whether or not the box touched it), every wire with **both** ends inside the set, and the buses, net names, and anchored labels riding them. A wire with one end outside is left behind — it would have to be cut.
3. Click the tab you want it on and press `Cmd/Ctrl+V`.

The design appears **under the cursor as a translucent ghost**, following it with no button held — the same way a fresh breadboard does when you pick one from the palette. Move it where you want it:

- Boards outline **green** where the design will land and **red** where it overlaps something already on that desk.
- Bring it near a board that's already there and it **snaps flush**, dovetailing exactly as a board you dragged into place would.
- **Click** to drop it. **Esc** throws it away and leaves the desk untouched.

The drop is all-or-nothing: if any board would land on top of something, the click is refused rather than pasting half a design and silently cutting its wiring. The whole paste is a single undo step, and the

pasted design is brand new hardware — fresh boards, fresh parts, and (for a ROM) its own fresh backing file, so editing the copy never touches the original.

The clipboard survives switching tabs, so you can paste the same design onto several desktops, or stamp it more than once on the same one.

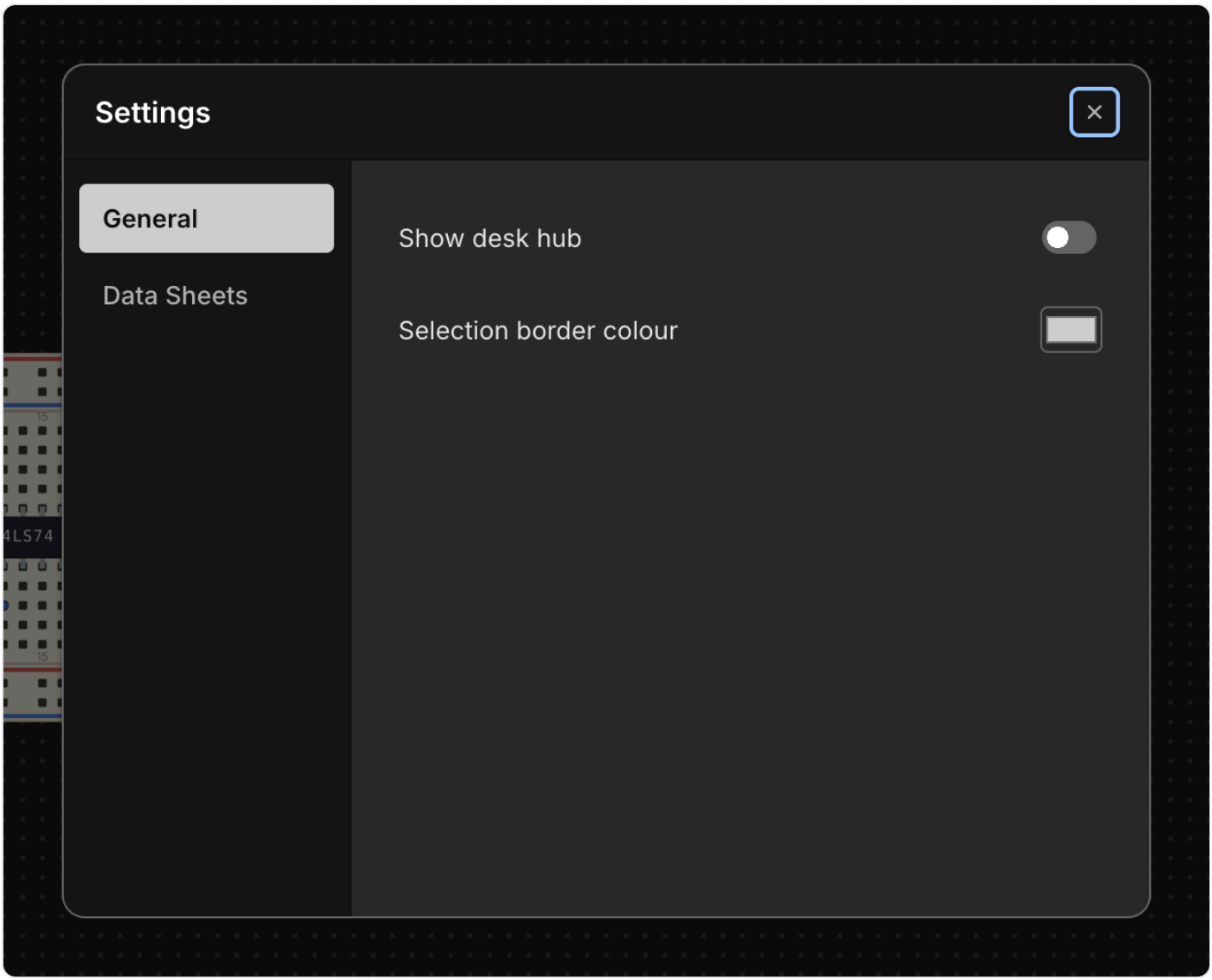
Leaving a project

A project stays open between sessions — named or not: relaunch Chip Hippo and it comes back on the desktop you were last on. Starting or opening another one asks first about anything you'd lose (see *Changing projects* above).

Closing the window or quitting asks the same question. Desktops are only written when you save them, so a tab still showing a dot would go with the window — Chip Hippo stops and offers to save it (or to name an untitled project) first. Cancel there and nothing closes at all.

Settings

Chip Hippo keeps its handful of app-wide preferences in one small **Settings** dialog — a tabbed master-detail card with a left nav rail and a panel on the right. It's deliberately minimal: two tabs, a couple of controls each, applied live the moment you change them.



Opening Settings

Open the dialog with `Cmd/Ctrl+,`, or click the gear icon in the top-right corner of the header. Both routes lead to the same dialog seeded with your current settings, so it doesn't matter which you use.

General

The **General** tab holds two controls:

- **Show desk hub** — off by default. This toggles a debug overlay (the `DeskHud`) on the desk; most users can leave it off.
- **Selection border colour** — a `#rrggbb` colour picker for the outline drawn around whatever's selected on the desk. Leave it unset to use the theme's default accent colour instead of a custom one.

Both take effect immediately — there's no separate Apply or OK step.

Data Sheets

The **Data Sheets** tab points Chip Hippo at an external folder of manufacturer datasheet PDFs on your machine. Click **Browse...** to pick a folder with the native file picker; the chosen path is shown next to it, with a trash-can button to clear it back to no folder selected.

Name the PDFs in that folder after each chip's catalog reference (for example `74LS00.pdf`). When a chip's file is found there, its pin-assignments window grows a button that opens the PDF in your system's default viewer. This is separate from the small datasheet crop the pinout window already shows for most chips — the external folder is for the full manufacturer document, not the built-in crop.

What else persists automatically

A few things aren't part of this dialog but are remembered between sessions without any action on your part: the app window's position and size, and the desk camera — your current pan position and zoom level. Close Chip Hippo and reopen it, and you're back exactly where you left off.

See Files, Autosave & Undo for how your circuit itself is saved.

Keyboard Shortcuts

Chip Hippo follows Electron's `CmdOrCtrl` convention throughout, so every shortcut listed here as `Cmd` also works as `Ctrl` on Windows and Linux. Most shortcuts are disabled while a text field has focus, and some are further gated on the current tool or simulation state — those conditions are noted alongside the shortcut.

Desk & simulation

Shortcut	Action
`Space`	Run / Stop the simulation (not while typing, or while a placement/wire/bus tool is armed)
`Cmd+R`	Run / Stop the simulation
`Tab`	Switch between the Breadboard and Schematic views (not while typing)
`Escape`	Unpin a probed net, then disarm the probe, then cancel a pending wire/bus, then cancel a placement in hand, then deselect — whichever applies first
`Delete` / `Backspace`	Remove the current selection (a part, wire, bus, annotation, board, or a whole marquee selection)

Tools

Shortcut	Action
`W`	Arm/disarm the wire tool
`B`	Arm/disarm the bus tool
`I` or `P`	Arm/disarm the probe tool (works even while the simulation is running)
`M`	Disarm whichever of the wire/bus/probe tool is currently armed
`1` – `8`	While the wire tool is armed, pick a wire color
`1` – `2`	While the bus tool is armed, pick the bus width (8-bit / 16-bit)

Placing & rotating parts

Shortcut	Action
`R`	Rotate or flip the part being placed or selected — see Chips & Components for the exact behavior per part type
`F`	Flip an LED's polarity while its placement ghost is armed
`Cmd+C`	Copy the selected part
`Cmd+V`	Paste a copy as a new placement ghost

View

Shortcut	Action
`Cmd+F`	Fit the whole desk to the window
`Cmd+Shift+F`	Zoom out to fit everything at once
`Cmd+=` or `Cmd++`	Zoom in
`Cmd+-`	Zoom out
`Cmd+0`	Reset zoom
`Cmd+A`	Toggle the Logic Analyzer panel
`Cmd+P`	Toggle the parts palette panel

Files & editing

Shortcut	Action
`Cmd+N`	New Schematic
`Cmd+O`	Open Schematic…
`Cmd+S`	Save
`Cmd+Shift+S`	Save As…
`Cmd+Z`	Undo
`Cmd+Shift+Z`	Redo

App

Shortcut	Action
`Cmd+,`	Open Settings
`Cmd+K`	Open the quick Keyboard Shortcuts popup
`Cmd+/`	Open the Chip Hippo User Guide (this guide)
`Alt+Cmd+I`	Toggle Developer Tools